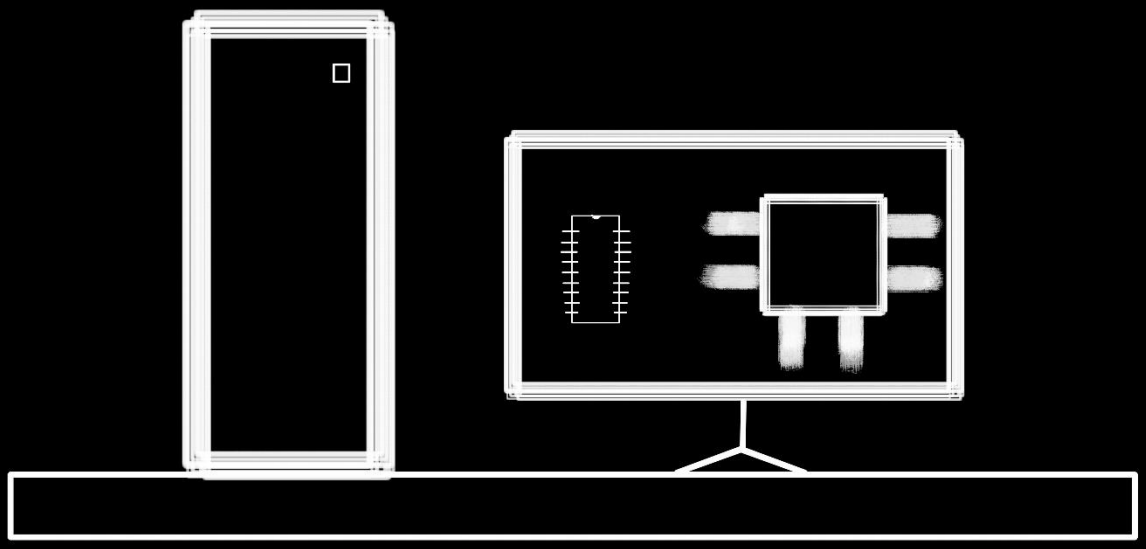
COMPUTERS

The Engineering and Programming Concerns, Concepts, Approaches and Implementations



ADSkeptiK (on Github)

The page is unintentionally left empty

The Structure of this book

The book is organized in subjects, chapters and sections. Each subject is to cover an extremely generic and vast CS subject that is the groundwork of many generic roles/jobs. We then have chapters which with the exception of chapter 0s specifically and in detail focus on job paths and projects for specific hardware used in industries, to get the user actually job ready. In chapter 0s we focus on getting the general concepts right and making the user ready for starting to then focus on the more specific routes. Then there are the sections for noticeable chunks of lessons that are related to a specific aspect of the chapters focus. Then there are Lessons which are taught in each section. For example: Chapter 0 of Networks focuses on the general computer networks stuff while the next chapter focuses on Cisco's specific routers or CCNA or stuff like that. This E-book was digitally written and uploaded by ADSkeptiK (Github) for free and is expected to be shared for free in it's digital form at least.

ADSkeptiK



Subject 1: Computers, Programming and Programming Interfaces

Chapter 0: Introduction

Before computers there was programming! The combination of **knowledge** in the form of noticing repeating patterns and **creativity** out of the love for saving time and increasing efficiency had our ancestors programming and automating chores since long ago. Time saving in spite of being the starting concern, became ONE of THE important concerns in programming. With the gradual growth of the societies,

civilizations and the domain of **knowledge**, the domain of possible applicable solutions expanded as well and alongside them the concerns over resource management, flexibility and etc also emerged. The realization of the importance of management resulted in understanding the important concept of "**System**" and the birth of this very concept as a logical solution. A system in plain terms is the **purposeful and ordered cooperation of a set of behaviors based on circumstances**. Naturally for a system to peak at it's purposefulness, that purpose should be prioritized in every aspect of it, pressuring all it's behaviors to be purposeful as well in the process. Each of those purposeful behaviors that make a system could be (and is) called a **program**, making a system also definable as a **Big Program** as a result. As our systemic and management thinking grew we started to replace humans within those systems with tools to save time and for systems to be safe of certain limitations that human resources would bring with them. It was these ambitions that ultimately led to the invention of **Computers**, tools capable and made for doing chores/running programs that humans used to do through saving and doing logical computations and eventually mathematical calculations.

Computations and Logic

In Theory

Computations which are the main point of computers, fundamentally have 2 undeniable and necessary requirements to work with:

1- The actual data that we want to use for computations (the Knowledge)

2- An algorithm/pattern/thinking patterns to compute those data with (The Creativity)

Since we live in a world that ultimately finds itself under the influence of the unknown, we could only prepare and ready ourselves to increase our chance of success for as much as our domain of **knowledge** and strength allows. Under these circumstances naturally:

- for data: the **preciseness** and **exact-ness** become the main but not the only ideals.
- for Patterns: **the higher chance of success ends up on the side of the one that is more ready for both a wider range of possibilities AND has the ability to handle them well**,

In order to get close to the ideals for these 2 foundations of computations, having **accurate definitions** becomes a very important factor in both. Accurate definitions are given by **accurately checking the standards** that are met (A standard is a complex condition based on a group of definitions that are based on units. A unit is a small simple measuring system defined in relativism to a constant). The best way to accurately check the standards (or to define them even) is to use smaller informative binary conditions that are either Met or Not Met, to decrease the ambiguity.

In Computers

The **condition based nature of computations** demands for computers to also be ready for **conditions and possibilities** to be able to fulfill their main purpose of existence. In order to achieve that and get, set and describe the data and patterns mentioned in the previous part **computers tend to use digits as they could be**

used for possession measurement , counting, calculations, Identification (describing data AND possibilities). We'll get more into what I exactly mean here in the first section.

The levels to programming

There are considerations for starting programming and then there are also layers to programming, from soldering literal hardware pieces to writing code into files on computers with the goal of implementing routines could all technically be regarded as programming. Regardless of which layer you wanna program in, one thing's clear: You need knowledge of the other parts that you'll need to use and deal with in that layer and you also need to come up with efficient plans to achieve the best out of what you have. We'll get to the general flow of programming and the important concepts in each layer for now and get to the details for implementing actual projects in each layer later. It's safe to say we're beginning from the very bottom and slowly going towards the top in the computer levels. The model below is a perfect way to represent what we're doing.

(Place Le Model pic here)

But without a clear first introduction to the important aspects of every interaction with computers we can't do anything yet. A programmer's performance is always directly influenced with 3 contributing foundations:

- The Computer itself

- The API(s)

- The Program(s) and their Input Data

Keep in mind we have abstractions in each layer but only the abstraction(s) that implemented specifically for programming the component(s) is called an API.

Computers

Any tool flexible enough to handle any level4 of computing could be regarded as computer. Obviously the scales, flexibility, speed and etc sensibly do differ from a computer to another based on which purpose or purposes they are prioritizing but ultimately, there are certain logically necessary conceptual roles that MUST and do exist in any computer:

- Some piece(s) of hardware to keep track of computations and data.

- Some piece(s) of hardware to manage and compute the data.

- Some pieces(s) of hardware to communicate with it's user.

These could exist in radically different forms but they have to exist.

What was once vaccum tubes became transistors and what was once just few transistors became an IC and then Microprocessors.In this layer you deal with micro architectures, soldering and etc.

To keep track of things numbers have always been the easiest way, hence we communicate with computers through them. Naturally having foundational understanding of numbers I'm gonna need to first re explain some common knowledge about them in a more powerful way that warms up your mind ready to understand things a lot easier from this point on.

Unique Symbol Set

A Unique Symbol Set (**UNSS**) is **a finite set of unique symbols** the members of which represent states established by the UNSS's specification specification.

E.g: UNSS_Decimal = {0,1,2,3,4,5,6,7,8,9}

Here the symbol 1 means a single instance of a thing, 2 means a single instance added by another and… . Keep in mind **a finite set of symbols does not necessarily mean a finite set of combinations and permutations of them**.

Amount

A UNSS's symbols themselves mean nothing until we conventionally assign values to them. Values are assigned to each symbol in the UNSS leading to the creation of **amounts** (**which are value-representing symbols**) and an **Numeric Foundation Set/ AMS (UNSS but now with value/ an AMS is finite set of symbols each representing a unique value)**.

Digit

A digit is defined as a limited finite container that could **ONLY AND ONLY** fit just a single drawer that's the size of a the **biggest Amount** in an AMS **(which in the conventional way we use the numbers equals to: numeric base-1) in t**.

E.g: D ϵ UNSS_Decimal

Here D is a digit, it could only contain a single member of UNSS_Decimal set at a time.

Digit Organization Conventions

The **Digit Organization Conventions** (**DIOC**) define how multi digit values are supposed to be organized and interpreted to give the context meant according to their purpose. A DIOC is a hypothetical ideal (usually unavailable) manual that tells us how to read and interpret digits, we tend to build this manual in our head by gathering chunks of information about the UNSS and the digit organization as the two subjects are usually intertwined.

The Standard Counting Convention

The Standard Counting Convention or TSCC is just one of the common conventions that we organize digits with. It's the most commonly used DIOC in real life for dealing with numbers but not necessarily the only one or the end to them. TSCC as the name suggests evolved alongside humans as a way to make possession measurement easier. The main idea behind it is simple: We count based and in relativism to a established threshold or **numeric base** for all digits.

Naturally we'll need:

1- A single decided **numeric base/amount threshold** for all digits.
2- A limited UNSS containing symbols to represent all amounts to be used for describing and measuring instances (**except a symbol to represent a filled threshold**). Typically in UNSS members there's a symbol for the **lowest amount of instances (LAOI)** AKA nothing (0 in humanity's case) and a symbol for the **just-below-threshold amount of instances (JBTAOI)**

(usually 9 in humanity's case). We intentionally do not define a symbol for the threshold itself cause the "symbol" of it and the counts above it are to be implemented using digital logic and relativism to the base.

3- Each time the amount of instances held by a digit exceeds/fills/reaches the threshold or threshold sized units could be extracted out of it, they are extracted as additions known as "**carry"** s to its left digit, these carries are according to the number of times the threshold sized units gets extracted. Any remaining amount of instances remains within digit.

The floating point convention

Each of the possessing states could possess just as many states within itself each of which could contain just as many and it goes on.

Digital Logic

As mentioned before (*in 1.0.Introduction*) we use numbers in the TSCC convention in 4 general ways:

- **Possession Measurement**: To determine/measure the amount we own/possess of something.(Measurement has 0 in it cause we could "have nothing)

- **Counting**: To determine how many of 1 existing object we have.

- **Calculations**: To keep track of changes.

- **Identification**: A number could be used to represent a unique possibility.

Digital logic is about at times using all these capabilities of numbers at times in parallel to their full extent to achieve and assist computations.

Numeric Systems

The number of different possibilities/states a single digit can represent is set by it's numeric system. A **numeric system defines UNSS and the numeric base**. Since in computers at the basic level we are really using numbers to measure the amount of data that the components "possess" or "own" we could also say the **Numeric Systems in CS set the threshold of the states of possession/count of unique possibilities that a digit could hold (which we'll call range count from this point on)** (yeah I know you probably understood nothing, but it was just an extremely short and beautiful way to sum it up, it sounded so good).

- For **Decimal** numbers it's 10 (You could possess from 0 to 9 states for each digit (**10** is the range count / number unique possibilities per digit)).

- For **Octal** numbers it's 8 (You could possess from 0 to 7 states for each digit (**8** is the range count / number unique possibilities per digit)).

- For **Hexadecimal** it's 16 (You could possess from 0 to 15 states for each digit (**16** is the range count / number of unique possibilities per digit)).
(Notice the word Count instead of value. It plays a big role later)

NOTE: Range count does NOT conceptually equal Base and it is possible to have and define numeric systems that have different range count and bases. They are usually the same because we start from zero in our common possession measurement usage of numbers (which is what we'll be dealing with 99% of the times) but just to be sure the generic formula for calculating it is:

***JBTAOI** – **LAOI or** Initial USS Symbol*

The formula calculates the actual distance of the initial symbol in UNSS till it reaches right before the base (or till it reaches the **JBTAOI** itself), giving you the actual certain .

Binary

For computers to compute:

1- They need to keep track of data, they do so using numbers (the 1st generic way of using numbers *as mentioned*).
2- As we said previously (*in 1.0.Introduction*) **binary conditions** are common in computations as they allow for (kinda) terminating ambiguity.

As a result of the two statements above the **binary numeric system** became a perfect candidate cause a **binary digit** AKA **bit** could be used in the places of any of those binary conditions to hold their state hence bit is the smallest basic unit of measuring the “vastness”, length and size of the possibilities in computers and since the said sizes in computers are usually at least in 10s of bits and above, Byte was adopted as a more common bit-based unit for the same purpose, each byte equals 8 bits. Aside from Byte the physics SI prefixes have also been totally repurposed in the field of Computers and CS to be used for bits AND bytes. In CS:

- Kilo = 2^{10}
- Mega = 2^{20}
- Giga = 2^{30}
- Tera = 2^{40}

All these repurposed CS SI prefixes work for both Bits and Bytes. That said the problem of ambiguity still very much remains in CS as believe it or not sometimes people use the physics defintion of the prefixes for CS as well. Some newer

prefixes such as kibi for kilo and a like have been defined to hopefully exterminate the confusion but haven’t become all that dominant and popular yet hence it’s good to actually read the manual of apps to see which is the one being used.

Decimal

The reason why the decimal numeric system seems a little weird compared to the average bit-based numeric system is that, in the average bit-based numeric system, the total number of states we can represent is determined and calculated based on the full, fixed number of states that each digit can represent. In the decimal numeric system, however, that threshold is actually decided externally, meaning we are not always allowed to use the full range and potential of our digits.

Octal

Hexadecimal

Numeric System Conversion

Digit Equalization

In order to convert numeric systems from numeric system A to numeric system B we need to first determine how many digits of numeric system B equals to a single

digit of numeric system A. This is achieved thru using $\log_B A$ where A is the range count of numeric system A and B is the range count of numeric system B.

Value Equalization

Our handling of the value conversion from one numeric system to the other is greatly dependent on 2 aspects:

1- Whether the target numeric base is smaller or bigger than the initial one.
2- Whether we are dealing with fractional or whole values.

Whole numeric value conversion

When we're dealing with a whole number If:

1- Target numeric base is bigger than the initial one:

    1- We sensibly divide the value to the target numeric system's range count.

    2- The remainder of each division becomes the value of the digit.

    3- The resulted dividened is the amount that has made it passed the digit and goes thru step 1 to 3 again till the number left cannot be divided.

2- When we're dealing with a smaller

IN DECIMIAL TO HEXADECIMAL CONVERSION WE CANT USE THE "JUST REPLACE" THE COUNTER PART IN HEXADECIMAL NOTATION AS HEXA DECIMAL DIGITS COULD STILL POSSIBLY HAVE CAPACITY TO BE LEFT FOR FILLING.

DURING BASE SAME BASE DIVISON IN NON DECIMAL NUMBERS WHERE THE DIGITS GO ABOVE 1 REMEMBER THE GOLDEN RULE OF "ONLY A FULFILLED CYCLE LEADS TO A CARRY".

The reason why the decimal numeric system seems a little weird compared to the average bit-based numeric system is that, in the average bit-based numeric system, the total number of states we can represent is determined and calculated based on the full, fixed range/number of states that each digit can represent. In the decimal numeric system, however, that threshold is actually decided externally, meaning we are not always allowed to use the full range and potential of our digits but only 10 states of them. This comes from the fact that decimal numeric system was born not based on bits but human biology.

Complements

In complements regardless of diminished or non diminished we swap them to the complement necessary to get them to their **in range threshold** .

Data Types

Due to the *mentioned reasons* the threshold of the size/length of the data we can contain is always finite and it's representation done bit based. According to the size of our data we deal with at the moment we use different sizes of containers from the finite range we have to hold our values accordingly.
The way numeric data saved in a datatype are to be read and saved (their encoding) is and must be different based on whether they're fractional or integral. The integral values :

1- Signed and unsigned integers
2- Signed and unsigned floating point numbers

Signed and Unsigned floating point

[sign][Mantissa]e[Exponent]

The number of bits for mantissa and exponent are fixed.

Boolean Algebra

The main use of Boolean Algebra in computers is thru bit vectors.

Bit Vector

Are strings of zeros and ones of some fixed length w. vectors according to their applications to the matching elements of the arguments. One useful application of bit vectors is to represent finite sets. Basically they could be interpreted differently according to the encoding used.

Vaccum Tubes

Transistors

Buses, Words

Buses are collections of electrical conduits that carry bytes of info between components. These fixed-size byte chunks are known as words. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

I/O

The I/O s typically connect to I/O bus using a controller or adapter.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Controller

Are chips in a device itself or parts of the motherboard. COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Adapter

A card that plugs into a slot on the motherboard. COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Endianness

This is one of those subjects that is usually stupidly explained… so now I explain it well so you wouldn't have to go thru the same shit I did to finally understand it

It's used to indicate the load direction of the sequence of input multi byte values that fulfill our n-byted buffers from their highest bit downward in a CPU architecture :

1- **The loading of these input bytes could begin from the first byte in front of them to forward (Big Endian or Most Significant Byte or MSB)**

OR

2- **the nth memory byte given to them backward (Little Endian or Least Significant Byte or LSB)**

---

*Many recent microprocessor chips are bi-endian.*COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

ISA

Processors operates based on a simple instruction set architecture AKA (ISA).

Micro Processors

CPUs basic operations are always in one of these groups:

Load: Copy byte/word from memory to CPU register,overwtiring previous val.

Store: Copy byte/word from register to memory.

Operate: Copy contents of two register to ALU to perform an arithmetic operation and then store the result in a register overwriting it's previous value.

Jump: Extract word from the instruction itself and copy it into program counter. Overwriting previous val. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

They execute machine instructions they get from a memory. They have a word size storage device (register) called program counter (PC) pointing tat the address of machine instruction in their memory. The processor reads instruction from pc and interprets and does the operations. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

ISAs

ARM

The ARM processors are usually bi – endian.

ARM microprocessors, used in many cell phones, have hardware that can operate in either little-or big-endian mode, but the two most common operating systems for these chips-Android (from Google) and IOS (from Apple)---Operate only in little-endian mode. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Intel

Most Intel-compatible machines operate exclusively in little-endian mode. On the other hand, most machines from IBM and Oracle (arising from their acquisition of Sun Microsystems in 2010) operate in big-endian mode. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Registers

Register File

Register file is a small collection of word size registers, each with its own unique name. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Cache

They serve as temporary staging areas for the information likely to be needed in near future. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

L1 Cache

L1 Cache on processor chip holds tens of thousands of bytes and can be accessed nearly as fast as the register file.

L2 Cache

L2 Cache holds hundreds of thousands to millions of bytes and is connected to the processor by a special bus. Accessing it is 5 times slower than L1 but still 5 to 10 times faster than memory.L1 and L2 caches use a hardware technology called Static Random Access Memory or SRAM for their implementation.

ALU

The ALU computes new data and address values

Micro Architecture

Defines the actual implementation of the processor and it's ISA

.COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Main Memory

A temporary storage device that holds both a program and the data it manipulates while the processor is executing the program, it physically consists of a collection of dynamic random access memory (DRAM) chips. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Amdahls law

Principle: The performance and time efficiency is directly based on the increase in the efficiency of components that contribute to it and the domain of said component in the whole system.

Formula in simple words: The new time performance of system equals to the sum of the previous time performance of the unchanged components and the reduced time of the improved component.

$T_{new} = (1-a)\ T_{old} + (aT_{old})/k$

$= T_{old}\ [(1-a)+a/k]$

In the formula:

- a is the domain of component.

- 1 is used to refer to the entirety of system to later be used for calculating the time of the parts outside the domain of component.

- k is the factor of performance, since here we're calculating the reduced time NOT the performance boost to it we divide instead of multiplying.

And then the formula for speedup becomes $S = T_{old}/T_{new}$

$S = 1/ ((1-a)+a/k).$

$0.2 + 0.8/k$

The APIs

Your programmer at the lowest level is your PCB or Solder.

Open GL

WinSock

Direct X

JDK

Microsoft Windows SDK

The Programs

When it comes to the representation of a program itself, dependency and reusability are some of the main oncerns.

Voltage(s)

Firmware(s)

BIOS(s)

OS(s)

In this layer you deal with Computer Organization.

Flow of Apps in modern systems

The machine instructions for an app are originally stored on disk. When program is loaded, they are copied to main memory, as the processor runs the program, instructions are copied from main memory into the processor. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

The processor fetches an instruction from memory, decodes it (i.e., figures out which instruction this is), and executes it. Thus we have just described the basics of the Von Neumann model of computing.

OPERATING SYSTEMS THREE EASY PIECES

Virtualization

OS's typically handle the efficient interaction, management and timing between components easier thru the use of an engineering concept known as **virtualization**. "Virtualization" abstracts the pain of hardware specific physical layer to make them look and appear as usable blocks to the user.

APIs

APIs allow using the "virtualized" hardware.

History

System V is a unix standard BSD is another.

Executables/Bytecodes

Chapter 1: Personal Computers

Chapter 2: Laptops

Chapter 3: Tablets

Chapter 4: Smart Phones

Chapter 5: TVs

Subject 2:

Programming

Languages

Chapter 0:

Common Concepts

Floating Point numbers

In case of using the scientific notation the mantissa’s evaluation could be modified using parentheses.

Rounding Up Numbers

The formula:((sizeof(n) + sizeof(int) - 1) & ~(sizeof(int) - 1)) is pretty much a well used familiar structure for rounding numbers within Microsoft's compiler definitions.

Datatypes

Abstractions such as datatypes are fully gone in the machine code and only exist as bytes and addresses.

Common logical errors

Off by one error

Since in loops we simultaneously use a number for identifying,counting and calculating. Things tend to get super confusing and more often than not the number of counts end up being more than what we had in mind. In order to overcome this obstacle remember these:

1) In loop counter we merely set the starting point of the count.
2) When the loop begins you are always already in the first count.
3) Considering your loop threshold is in positive numbers and it goes thru incrementing. The formula (1- (counter starting point)) , marks the offset of the actual count of your loops from the desired.

Interpreters

Compilers

Chapter 1: The C Programming Language

C was introduced and is used as a compiled programming language… it's it's whole premise. Just like any other programming language, knowing the syntax of this language is one thing and the way we tend to program in it in jobs is another and the ready to use library it has is another,hence we start by getting straight to the programming approaches in this language, followed by explanation for the basic everywhere stuff such as syntax and it's keywords and finally the library.

General flow of C programs

Application Building Process

Programming in C tends to be made of the following steps:

1- Source Code Modularization: The C source code is modularized by putting the definitions of related functions, structs and global variables in files with descriptive names for the side of program that they implement, followed by .c extension, this way each C file becomes a module of our code.
2- For each module (.c file) we create a header file with the same name as the module but followed by a .h extension instead, in which we place the declarations for all the definitions in the module.
3- The .c files are then given to C compiler as input. The compiler first processes the Preprocessor directives (parts of C code).
4- The preprocessed files are each checked to make sure the declaration of each function, structure, variable is present in the file **BEFORE** it's place of usage in code. The passed files (called translation units in this stage) are then turned into their respective machine code files (AKA object files usually with the .o or .obj extension). The (the single) entry point of the program is found and "marked".
5- The linker resolves the calls to functions in other object files, links/stiches them all up together, looks for the (the single) entry point of the program and

sets it as the beginning of the code and turns them all into an executable binary, a program that could be run by the OS or a static or dynamic library (both of which we discuss in the section).

CRT (C Run Time) What is it

CRT is a block of code that's added before the entry point and is responsible for calculating and "saving" command line arguments and possibly assigning the main's parameters to them according to the standards (such as the case with argc and argv).

The Programming Process

Libraries/APIs

These are essentially groups of functions, structures and etc written by other programmers that we use in our applications. These libraries come in 2 forms.

1- Static Libraries: Compiled object files of function definitions to be linked at linking stage.
2- Dynamic Libraries: Compiled files to be accessed at runtime.

Regardless of which kind the library is , the code of the program using them obviously needs the declaration/prototype of the functions defined within them for compiler not to nag in pre linking stages. For that reason we always need the "include" header of these libraries that's including just the very same prototypes.

It’s in the very same header that we implement the conditional inclusion and conventions to make sure only the right declarations are loaded for the functions of the library and are therefore “accessible” throughout the code.

The Entry Point

Generally the default entry point of C programs is to be their **main** function (although certain APIs could and DO demand a different function names with different and varying numbers and types of arguments as their entry, (an example being Win API) which is exactly why you’d check the documentations regarding the implementations of C in said APIs before jumping into programming but for now we’ll stick with the general API independent main entry that’s expected to work on most systems), main is to return int and either have no parameters (in which case the parameter should explicitly be written void!) or have parameters int argc and char*argv for systems with command line environment support for C programs. The reason for this is that the process of running the executables of C programs begins by first setting the command line arguments (argv and argc) thru an initializer program called C Run Time that then calls the main function. The first argument in systems with command line environment support for C programs is assigned with the number of commands given as input to the standard input stream in the stack “before hitting ENTER” and the second argument is assigned to holding the string array that contains those arguments.

Build Specification

Before clicking on Build the configuration (debug or release) for build alongside the architecture and project dependencies (possible reliance on other projects to be built) is set and then the building begins.

Modularization and cross platform-ability Approaches

Modularization and cross platform-ability approaches could take place in 4 stages:

1- ***Preprocessor Stage (Macros)***
2- ***Source Code Stage (Encapsulation)***
3- ***Compilation Stage (Translation units)***
4- ***Build Stage (libs and Usage of smaller exe s thru Command lines)***

Preprocessor Level

The modularization in this level decides the **inclusion of the right chunks of code according to system's parameters**. Cross-platform-ability of codebases is typically implemented within this level. Some very known methods used in this level are:

1- ***Label Conditionalization***: Compilers and libraries of code are typically written to conditionalize a list of system parameters as labels for usage in ***conditional inclusion***, the definition of these labels (using the #define LABEL or compiler settings) later leads to the inclusion of certain corresponding code chunks after preprocessing. 2 best examples for this are

the _UNICODE and UNICODE labels included in MSVC C/C++ empty projects by default within the Preprocessor settings of the projects. _CRT_SECURE_NO_WARNINGS is another example.

2- ***Conditional Symbolic Function-Overload Abstraction***: (It's a self made long name but a descriptive one for a common method which I'll refer to as **CSFOA** from this point on… It's still more descriptive than shit like Virtual Private Server!): There are times where a systemically generic function would have multiple system-parameter-dependent versions (such as the generic systemic functionality of reading character which could have it's 1 byte character and wide character versions or a GUI function that should incorporate different infrastructure on Linux than it does on Windows), it's in these times that we use method 1 to define the name of the systemically generic function to be replaced by the name of the right "overload" version of it.

3- Name convention implementation: Naturally the possibility of name conflicts in C is generally higher due to the absence of some really helpful features such as classes, function overloads and custom namespaces that were added in C++. With all the said options out of the window using macros for adding prefixes and postfixes to function names is the way that's been used to enforce naming conventions on them in C.

Header Files

Header files are used to include the declarations for functions and types and keeping track of them till they are resolved. Putting the info that's likely to change

Source code level

Functions

Compilation Level

Translation Units

At this level modularization happens thru the use of .c files. The definition of functions and structs that depend on **similar infrastructure** or **help in fulfilling a single generic goal** (usually **both**) are placed together in a single .c file (right after the inclusion of the declarations for their required infrastructure within the very same .c file of course). Naturally the ideal for efficiency in this level is to try to avoid including unwanted declarations by our .c files. We can get closer to this ideal from 2 aspects:

1- By getting the most out of each header inclusion: Through placing functions that use the same infrastructure module we assure that we get the most out the included header in .c file.
2- Using Optimized headers: Include a custom made header that ONLY contains the needed declarations.
3- Manual Forward declaration: Selectively and manually write only the declarations for the part of infrastructure module that's actually used in our module, on top of the .c file.

Ultimately since in C each .c file becomes a translation unit it's best me make sure code is distributed among .c files in a way that changing a mere part of the program wouldn't need or demand changing or recompiling its entirety but the translation unit we have changed.

Header Only Libs

Sometimes folks may put the entire definition of functions in headers instead of just a declaration. Such approach could be regarded as bad practice for when functions are likely to change, as it shows little flexibility for changing functions in future, logically demanding the recompilation of all translation units that have included the header in one way or another for any and every change.

Link Level

Usage of Static Libs

Static libs are archives of **object code** s

Usage of Dynamic Libs

Build Level

Static Libraries (.lib)

They are binaries following a structure that linker can use for relocating the references or connecting them to DLLs.

While linking,these libraries either have one header that includes the declaration for all the functions within them, or the number of nested headers that do so next to the static lib file.

WARNING: It’s very important for the static lib to have a simple name. Avoid bullshit, spaces and ‘ and odd characters as much as possible as I personally ran to a configuration parameter error related to “Additional Dependencies” setting in MSVC/C++ while trying to link my static lib once.

There's little difference between machine code object files and static libs but the difference is noticeable nevertheless more specifically in windows. Lib files are archives of those machine codes and translation units in single respective libs.

Regardless of the format of static libs, the logical structure and accessibility to them is a matter of importance. In order to access a library the client of it needs the declarations of the functions or at least the declaration of the functions they need to access. While from outside the cleanest least painful approach would be to have the declarations for all the contents of all the translation units in the lib, written in a single header to then include and use them; the same "ONE BIG HEADER with ALL the declarations" approach doesn't look all that good while developing the library, and it only shows itself significantly when the size of the actual header gets big. It's lack of modularization.

Which is why I've personally come to prefer the header modularization approach for development and then the single big header for accessing library approach. Although again depending on situation these approaches could get different. Such as the case with Windows API for instance. So yeah it's a whole other level of engineering on it's own as well, often not described at all in computer engineering or CS courses for some reason (Yeah there goes CS for you).

Dynamic Link Libraries (.dll)

The compiler behavior

Default status of const values

The C compiler tends to see **values** as constant signed by default and any time it's not stated to be unsigned.

Read Only Data

Values such as string literals are saved in the read only section of the application and regarded as const. Any attempts to change the rodata value by the app becomes an access violation.

Static/Dynamic Memory Allocation

Pointer casting

Pointer casting can allow us to view bytes differently tho not advised.

The Language

The Essence of Data

Objects

The pieces of memory we use to hold data.

Constants

Constant Literals

Long Integer constant literals

A long integer constant is written with a l ir L.

Floating-point constant literals

Floating-point constants contain a decimal point(1.2) or an exponent (1e-2) or both, **their type is double unless suffixed**. The suffixes f or F indicate a float constant. l or L indicate a long double.

String constant literals (string literals)

Technically a string constant is an array of characters. The internal representation of a string has a null character '\0' at the end , so the physical storage required is more than the number of characters written between the quotes.

If we initialize a character pointer to a string literal it basically gives the address of the first character of that literal. **The String literal to the compiler just seems as the address of the first character.**

STRING LITERALS FOLLOWED AFTER EACH OTHER WILL BE CONCATENATED.

e.g: "sda" "sda" == "sdasda"

WE CAN ALSO WRITE A SINGLE STRING LITERAL SPREAD BY A SINGLE FORWARD SLASH EVERYWHERE THE LINE ENDS

e.g: "sa\

sa"

==

"sasa"

"

Symbolic Constants

Named constant literals using preprocessor directives.

Constant Variables

The constant variables are actual objects.

Data Types

All data types are ultimately broken down to bits. With data types we merely define the number of bits and how they are to be treated.

Basic Types

int32_t

uint32_t

int64_t

uint64_t

char

A typically 1 byte data type that can hold upto 1 byte of values signed and unsigned representation of values is determined by the keywords and by default it's signed. It can be assigned with both variables and literals of It's own kind and the integer kind so long as the integer is within the 1 byte range.

Character Constant Literal

These literals are one byte signed integer literals represented between single quotes. A character, Octal digits and Hexadecimal digits could be used to indicate the value that's meant by them.
Using a single character would indicate the ASCII value of the character:

e.g : char c = 'D' //Means take the ASCII value of D and give it to the variable.

String literals

Are shown in " "s and their to code they represent the address of the starting member of their array of characters. If you put two of them together to C they seem like a single concatenated one.

e.g: "ds" "ssd" == "dsssd"

Also you could break a single one to a multiline one using the following approach:

E.g:

"sdsds\
sdsd"

Pointers

Pointers to functions

The format is this:

functionReturnType (**FunctionName*) (*paramType1 paramName*,*paramType2 paramName,...*)

you can even have an array of functions:

functionReturnType (**pointerToFunctionList[numberOfCells]*) (*paramType1 paramName*,*paramType2 paramName,...*);

Which could later be used to invoke a set of functions like:

(**pointerToFunctionList[index])(paramType1 paramName*,*paramType2 paramName,...*)

Pointer Casting

In order to cast pointers use the following notation:
(casttype *) pointername;

User Defined Types

Arrays

If there are fewer initializers than the size of array,the missing elements will be initialized to 0 by default. By definition, the value of a variable or expression of type array is the address of element zero of the array. EACH ARRAYs POINTER OFFSET ARITHMETIC IS BASED ON THE SIZE OF THE TYPE THEY CONTAIN NOT THE THE SPACE THEY ALLOCATE THEMSELVES E.g : a

STRING pointer allocates 4 bytes but the individual members of the array are offset by 1 as the type that the string is containing is a char.

IN C multidimensional arrays are contigious.

Struct

Use/print or etc of uninitialized struct variables is an error in C.

Union

union is basically a single variable that can hold any of the types within it.

Enums

Enums are one structured way to define constants. It's a place holder, an object of which's kind can hold a list of Symbolic constants (which are integer values) as values. It's values are regarded and **could be used and treated interchangeably as integer constant literals.**

They are defined like this:

enum enumerationName {constantName1,constantName2};

Unless explicitly initialized by user within the definition, the values of each constant start with 0 for the left most constant in its list and go up by 1 as you go right ward. We then instantiate enums using the following format:

enum enumerationName instanceName = EnumConstant;

The constants defined within enum are available without any necessity for scope resolution operators.

For every enum, the compiler generates the static method **values** that returns an array of the enum's constants in the order they were declared.

Typecasting

Integral Promotion (flawed explanation)

Integral promotion: If an int can represent all of the values of the original type, then the value is converted to unsigned int otherwise it's converted to unsigned int. The process is called integral promotion.

Operators

Operators are used for managing the use of data. They have an order of precedence to which we must pay attention while using them to get our desired effects.

Binary Operators

Used to mask off bits (for shit like rounding up numbers).

Conditional Operators

!=

The precedence of != is higher than that of =.

Ternary Operator

? operator

This operator is used for conditional existence of a piece of code where we can't use if. It kinda behaves like a conditional inclusion preprocessor whose life time continues after the preprocessing stage.

Bitwise Operators

! Operator

It's sensitive to zero and turns it to 1 otherwise turns it to 0.

Definitions

Chain Assignment

We can perform chain assignment in the following form:

int var_a=0, *ptr=NULL,var_b=-1;

Declarations

Control Statements

for loop

The iteration variable always ends up at the very condition. That's why unless you're careful, you define it inside the parenthesis.

2 things to keep in mind regarding for loops:

1- The loop counter SHOULD and consequently does end at the fail condition. Which is why it's advised to define counter within the parentheses as otherwise you MIGHT forget this and use the fail value-ed counter unless you're aware of what you're doing.
2- The loop counter pre-counts at the beginning of the loop, this means that if the condition breaks in the middle of the current cycle, the cycle is still counted. This could cause trouble if you don't pay attention.

My ultimate idiom for you to keep in mind for getting the loops right is: "Make sure the count only begins after the passing of the first cycle".

The main concern and challenge of for loops is that counting them is done as a part of the cycle itself which is why many things could go wrong naturally the number of cylces will never equal the number of increments for the counter.

Another idiom: "Continue the cycles and adding one **until** we reach the **count** we want it meaning the loop goes on **till before** the count we want otherwise it will go above it"

Functions

C functions don't allow overloads. If the return type of function is omitted it is assumed an int. The value that a function returns will be converted to the return type specified if necessary(what did I even mean here, check notes from K&R!). When a function calls itself reculsively each invocation gets a fresh set of all the automatic variables,independent of the previous set. Recursive code is easier to understand and is especially convenient for recursively defined data structures like trees.

Headers

Preprocessor Directives

These directives affect the source code in the source files before compilation.

They could be one line or multiple lines in order to force them to consider the next line use a backslash \

Note: There shouldn't be anything other than a newline character/enter followed after the backslash, even a space after it would cause error.

#include

Is one of the many directives used for including header files. Literally copies and pastes the files in front of it in it's location. It could be used in 2 ways:

- #include <filename.h>: Searches the compiler's include directories for the header.
- #include "filename.h": Searches relatively to the address of the source code it's defined in.

#define

Is used for defining symbolic constants and macros.

Using # in a macro's replacement text would mean the followed parameter is to be regarded as string literal. Using## literally concatenates the preceding token with the following one.

#if

#ifdef

#ifndef

#endif

Static

Static keyword can be used for functions and variables and would cause a different behavior for each:

Static Functions

By defining functions as static, their definition only becomes ONLY visible to the file that they are defined in(meaning even if you include their declaration in other files they could not be linked to any definition, resulting in undefined link error or the use of another available definition with the same name).

Static Variables

Defining variables as static could cause 2 different behaviors in them depending on the scope their scope:

- For global variables: Defining them as static makes them only visible to the file they’re defined in.

- For local variables: Defining them causes them to keep their value since the last time their function was invoked.

External static variables thus provide a way to hide the names of global variables in a source file.

extern

external variables

An external variable must be defined exactly once, outside of any function while the declaration may be an explicit extern statement or may be an implicit from context. Before a function can use an external variable the name of the variable must be made known to the function.One way to do this is to write an extern declaration in the function; the declaration is the same as before except for the added keyword extern. Array sizes must be specified with the global arrray's definition but are optional with the extern declarations.

Expressions

An object is a named region of storage: an lvalue is an expression referring to an object.

Conditions

If and else

STUFF OUTSIDE OF ELSE BLOCK WILL HAPPEN EITHER WAY.

Typedefs

We start typedefs by capital letters to make em standout.

Apendix A: Data Types

integer

Apendix B: Functions

Apendix C: Headers

Apendix D: Preprocessors

Apendix E: Scopes and Storage

The Standard Library

Headers

stdio.h

Contains the typedefs for many types thru the inclusion of other headers which at the end rely on OS specific headers. In Microsoft's Windows this reliance is on vcruntime.h which is indirectly involved and included in many if not all headers.

stdarg.h

va_start()

Is a function/macro used in functions with variable length argument lists, responsible for setting it's first argument to point at the first unnamed argument of the function, using the second argument given to it which should be the identifier for the last named argument in function parameters.

va_end()

Cleans up the function variable length argument list.

function

fgetc()

The flexible version of fchar capable of reading characters from streams (files) other than the standard inputstream(also known as stdin) which was the file that getchar() read from.

fputc()

The flexible version of putchar capable of writing characters to streams (files) other than the standard outputstream.

feof()

Checks to see if the EOF marker for the given argument of it has been reached.

*int main (int argc,char*argv[])*

In environments that support C you can pass command line arguments to the program when the main is called the first argument which is int argc is the number of parameters

(set by 1 by default and calculated by the command line environment at first) and argv is the argument vector which is an array of pointers to characters and has at least one argument (it's 0th cell which is the name of the program).

scanf()

The function takes a list of conversion specifiers. After parsing each, it decides whether to see the bytes as an ASCII byte. It takes as much length as the first argument has defined and then any extra values will be given to the following args.

Data Type	scanf conversion specification
double	%lf
long double	%Lf
float	%f
unsigned long long int	%llu
long long int	%lld
unsigned long int	%lu
long int	%ld

unsigned %u
int

int %d

unsigned
short %hu

short %hd

char %c

FILE

Is a file structure containing the info that program needs to process the file.It may or may not contain a file descriptor, an index to an array in OS containing additional settings on handling file.

stderr

standard error stream.

stdin

standard input stream , can be used with putchar and getchar.

stdout

standard output steam.

fgetc

The function **READS** a character from the file given to it as argument in the form of a pointer

fputc()

The function **WRITES** the character in its first argument to the file specified by the FILE pointer in its second argument.

fopen()

The function is used to create a file with the name(extension included) of it's first argument for the mode defined in it's second argument. It returns the address of it as a FILE pointer.

feof()

The feof function's argument is a FILE pointer to the file to test for the end-of-file indicator—stdin in this case. The function returns a nonzero (true) value when the end-of-file indicator has been set; otherwise, the function returns zero (false). This program's while statement continues executing until the user enters the end-of-file indicator.

fprintf()

Writes into the stream given to it's first argumnent.

sprintf()

Print f but prefixed with a new first arg that acts as an output buffer for printf.

malloc()

Takes the number of bytes to allocate and returns a void pointer to the chunk of array of the number of bytes it's allocated. It is okay with the number being a

variable.We typically mix it with the sizeof function and a pointer type casting so that the memory is as much as needed and accessed accordingly as well:

```
userdefinedtype* userdefinedPtr= (userdefinedtypename*)
malloc(sizeof(userdefinedtypename*n);
```

Error Reading

The function is declared but not defined

Causes: Possible use of static function in a file other than the module.

The argument in additional dependencies was supposed to be Boolean

Possible spaced or ‘ ed naming while defining the dependencies.

Chapter 2: The C++ Programming Language

General flow of C++ programs

Entry point

Modularization

Forward Declarations

We use forward declarations to avoid the Header inclusion in header scenario, which could make the already bad dependency that the headers introduce, EXPONENTIALLY worse and more costly.

In this method we merely declare the types used in a class's header and then in it's definition C++ file include the header that contains the definition for those types.

C++ is C with extra capabilities and headaches as well. Read the C language before this so that I can only focus on the C++ specific stuff here.

It's important to keep in mind that forward declarations wont work for classes that are being inherited or any instantiation… So we can't make instance

variables of the type. As a result we neatly use pointer instance variables to manage such objects when going with the forward declaration method.

Headers

For subtle reasons that we explain in Section 23.4, headers should not contain using directives or using declarations

Polymorphism

Keep in mind the pointer to inherited class only works for infrastructure already defined within the base class.

The Language

Casts(reinterpret cast)

Scopes

Global Scope

In oder to refer to global scope we just use :: in C++.

namespace

Loops

Range-based

Used to do a certain piece of code for all objects of a certain kind in a certain container. They follow the syntax:

for(type name:typeContainer)
{
Do this
}

These loops actually only support the STL arrays and the built in ones NOT the dynamic ones as the two types just mentioned have.

Functions

Default parameters

A member function could have default parameters which **SHOULD ONLY BE DEFINED IN THE DECLARATION**.

Const parameters

For a member function that is to treat an argument as const the const keyword should be present before the definition of that argument **IN THE FUNCTION DEFINITION , since C++ is not as type sensitive to matching in function arguments' qualifiers matching in declarations and definitions, as it is to the functions qualifiers themselves which is "odd"**.

Function Overloads

An existing function could have different parameter lists while sharing the same name each of the definitions with those different parameters lists is an overload.

Function Template

Is used to define a generic form of function to save the time of overloading. A function declaration has to always prefix a template parameter list both in it's declaration and definition like the format is like below:

template <**typename** type'sname>

function prototype or definition

the type'sname officially becomes a Generic type.

Inline functions

Encourages the compiler to replace the body of the function at the location of calling it.

You can’t have an inline and non inline version of a single function simultaneously.

Overloaded Operators

Are functions used to replace the functionality of existing operators for certain arguments.

The syntax is typically like this:

ReturnType **operator***theoperator(paramtype param name, paramtype2 param name2,…)*

The first operand to the operator becomes the first argument.

The delete keyword

The delete keyword causes the function to ultimately make the function signature unusable. (further questions needed).

It’s usually assigned to function in the format:

Function()=delete;

Making it unavailable.

Classes

Access Modifiers

Define what can and cannot be accessed directly from an object of the class. By default anything defined in a class is set to private. You can change access modifiers mid class which is not recommended. The general structure is that public side is defined at the top of the class and then followed by the private side.

Public

Private

Class's Type Definition

Class definition in C++ happens in the following format:

```
class classname{};
```

The general way for defining a class is to have the declaration of it (which only contains the member function declarations and the variable declarations) in a .h file and then go on to define the actual member functions in a .cpp source file with the same name as that header, after including the header containing the declarations.

Static and Non Static member Functions

Member Functions

A class's member functions could be static and non static with the exception of the **constructor**.

Class's Member Function Definition

Member functions of a class are defined in to ways:

1- Fully defined the within the Type Definition of the class with the following format where you SHOULDN'T put semicolon after member function definitions.:

```
class classname
{
returnType memberfunctionName(type param1,…)
{body of the function}
};
```

2- Prototyped in Type Definition and later defined in modules: In this approach With the assumption that the type definition is presented and has the prototype of the member functions within it, we can use the :: scope resolution operator to define the member functions in detail in the following format:

```
class classname

{

function1 prototype;

function2 prototype;

};

Returntypeforprototype1 classname::function1(…){}
```

Const Member functions

Defining a member function demands it NOT to touch the instance variables of the class. Using the following format:

ReturnType FunctionName (type param1,type param2,…)*const*

The *const* should be present in both the definition and the declaration.v

Virtual Member functions

Virtual functions are a way to abstract away certain abstract functions, the definition of which could vary based and according to the inheriting classes.

In order to define such functions which have the ability to be redefined in child classes.

Example:

Baseclass.h:

Virtual void foo (void) =0;

InheritingClass.h

Void foo(void) override;

InheritingClass.cpp

Void InheritingClass::foo (void){}

A virtual function could be :

- Pure: If you add the = 0 after the parameters. In this class somewhere in the chain of instantiation for your object, definition for that function has to be created.
- NonPure: If you attempt to define it at the class namespace (inline or on separate file), providing it with a default version to use in the case of inability to find an actual definition for it.

You could override virtuals of the parents of inheriting classes all you want.

So long as there's a virtual function within the ancestor tree of the inheriting class you could override it.

Member Operators

By default the member operators demand that their first or the only argument(the left operand) be of the class's type.

Copy Assignment Operator

Is essentially a special member operator which due to it's special nature demands a certain signature/format (just like a **Copy constructor**) which is:

classAsReturnType operator=(classname& class)

Copy Assignment Operator is detected and run upon any assignment between two class objects <u>THAT IS NOT AN INITIALIZATION.</u>

Move Assignment Operator

Is essentially a special member operator which due to it's special nature demands a certain signature/format (just like a **Move constructor**) which, in order to have a different signature than **Copy Assignment Operator** demands the following format:

classAsReturnType operator=(classname&& class)

The necessary and demanded change in its format makes it only accept rvalue references as argument by default. Consequently, the Move Assignment operator is ONLY invoked by default upon the assignment of an object with another Non-named temporaray object (e.g: classname a{classname{}} but it CAN absolutely be manually run upon initialization with an existing named object through the usage of std::move function in the following format std::move (*initializer object); function***.

Conversion Operators

Temporarily converts an object of it's class type to another type. **The defining feature of these operators is that the compiler can call these functions implicitly to create temporary objects.**

Friends

Constructors

The short version: A constructor is a member function to be called right upon instantiation of a class, usually to set some of it's variables at that very moment.

The long version: A constructor is an **essential role** for a class's non static member functions, that one non static member function must ultimately play. The way the constructor is decided varies depending on which of the main 2 circumstances we are dealing with:

1- No candidates for constructor : In this scenario there are no suggested functions for the role, hence the compiler automatically generates a default empty constructor at compilation. The constructor created by the compiler using this method is called a **Default Constructor**.
2- There's a custom list of candidates for constructor: In this scenario the user has defined a constructor or overloads of constructors for different circumstances. In this scenario the compiler will either force you to meet the required circumstances for at least one of the overloads of the constructor and then call it OR if the required circumstances are met it jumps to calling the constructor that fits the situation. The constructor that's decided using this method is also known as **Custom Constructor**.

There are certain requirements that a custom constructor (or a candidate for it) **MUST MEET**:

1- It **SHOULD NOT HAVE** a return type.
2- It **SHOULD NOT BE** static.
3- It **SHOULD** be a member function.
4- It **SHOULD** have the same name as it's class.
5- It **SHOULD** initialize the const members of the class using an initializer list if they do not have default values.

Constructor initializer list

Constructors typically initialize member variables within initializer lists. This happens before the execution of their body.

The initializer lists are supposed to be defined within their definition.

Format:

Date::Date(unsigned int mn, unsigned int dy, unsigned int yr) 13 : month{mn}, day{checkDay(dy)}, year{yr}

You are mandated to run the constructor of the inherited class within the derived ones initializer list.

Constructor Delegation

There are times when the thing that happens in one overload of the constructor also happens at the beginning of another. In such times constructor delegation is one good approach for code reusability. In short: Constructor delegation is when a constructor overload calls another constructor overload to handle the identical first step present in it with given arguments, instead of completely redefining that first step. It's achieved using the following format:

```
constructorName(params): constructorname{params for the constructor that runs first} {The of the constructor to run after the overload was run with those parameters}
```

Copy Constructor

Copy Constructor essentially allocates new memory for the corresponding object and copies the values of corresponding object into them.

The signature format for them is:

ReturnType className(className &)

This is the main format for them.

The Copy Constructor is ONLY invoked by default upon the initialization of an object with another named object. (Careful NOT to mistake the copy constructors with the assignment operator overloads which happen according to lvalue's overloads upon any assignment that's NOT an initialization!)

Move Constructor

Move Constructors essentially "steal" the members of the other object thru pointers and object addresses. Since the one parametered reference-to-class format is already the signature format for Copy constructor outta the necessity and the usage context of the Move constructors their format has become different.

ReturnType className(className &&)

The necessary and demanded change in its format makes it only accept rvalue references as argument by default. Consequently, the Move Constructor is ONLY invoked by default upon the initialization of an object with another Non-named temporaray object (e.g: classname a{classname{}} but it CAN absolutely be manually run upon initialization with an existing named object through the usage of std::move function in the following format std::move (*initializer object); function*.

Conversion Constructor

Another special form of constructor that doesn't actually work like a constructor at all in the sense that much unlike all the other kinds of constructor, it's invocation is

not limited to the moment of instantiation and initialization, **A conversion constructor, is actually more like a conversion operator that instead of changing the class type to other existing types, turns the other existing types to class type**. It takes a single parameter and turns it into the type of its class. With the exception of copy constructors, **any constructor that can be called with a single argument and is not declared explicit can be used by the compiler to perform an implicit conversion** which is why we use the keyword **explicit** to make.

Class's Object Instantiation

the instantiation can but does not always happen after the class definition:

```
class classname {}instancename;
```

More often than not the instantiation happens like a normal variable definition (Assuming the class's declaration and definition have been seen linked):

```
classname instancename;
```

We typically initialize a class (that is, it's constructor) by {}.

Instance Variables

Statics

Static variables are global variables merely within the calss's scope. Naturally it's stupid to do literal initialization of them within the class. Leading to fucked up behavior and compilation error if you define them as const and try to initialize them within obj. We could define static instance variables outside of the class

declaration as they are unique in the sense that they really aren't instance variables, granting them the ability to exist and be defined regardless of instantiation.

Inheritance

Constructors

Inheritance has to account for constructors as well. Obviously in base classes with default constructors, this doesn't seem like a big deal but for those with **const** members and those with custom constructors this is an additional pain in the ass indeed. The way we account for this is through using an initializer list to delegate constructor to the base constructor first.

Delegation

Calling the base class constructor

In this situation you can both "delegate" constructor AND have your initializer list after it using

```
class A {
public:
   int x;
   // COnstructor initializing
   // the data members
   A() { x = 0; }

   A(int a)
   {
      cout << "Constructor A(int a) is invoked" << endl;
```

```
        x = a;
    }
};

// Complex class
class B {
    int data;
    A objA;

public:
    // COnstructor initializing the
    // data members
    B(int a)
        : objA(a)
    {
        data = a;
    }
```

Constructors in Composition

You could and should call the custom constructors for the existing objects within the derived class IF they have any AND are not to be initialized by Copy constructors, so that their initialization can happen.

Constructor Inheritance

We inherit a constructor with a :

using BaseClass::BaseClass;

inheritance, which results in the baseclass's constructor being defined as an overload constructor in the inherited class.

Preprocessors

extern "C" {C function declarations}

Makes C++ compiler load C function declarations accordingly.

__cplusplus

If the compiler is a C++ compiler.

Data Types

The main types are exactly like in C.

Reference Type

The reference types in C++ are more like a role. They are automatically dereferenced, const pointers to data.

THE BEST SPECIFICALLY POSSIBLE WITH REFERENCE TYPES MOMENT FOR THE REFERENCES (THEIR SHINING MOMENT) IS IN RANGE BASED LOOPS.

new type [ ] [ ]

Dynamic Memory Allocation

new

In C++ the keyword **new** is used to dynamically allocate memory.

delete

We use **delete** for to deallocate memory for a variable and **delete[]** *arrayname* to delete an array. Doesn't work for non dynamically allocated memory.

Scoped enum

Scoped enums are basically C's enums but with an extra level of type checking.

They are defined like this:

enum class Name {constant1,constant2…}

The main difference is here the constants to the ompiler won't seem as integers much unlike in C's enums and now they are of type Name in order to assign each

instance of these enums with one of of their constants we must now used the scope resolution operator and their name like:

Name::

STL

Types

std::string

The type introduces dynamically allocated character string type and goes on to implement + concatenation operations for it and all. The + concatenation begins since reaching the std::string type and will continue on.

std::cin

>>

An overloaded operator that takes a first argument of type std::cin (left param) (which is a whitespace terminated input stream) and gives it to a generic type second argument.

std::array<type,cellcount>

Are fixed-size collections of data of same type. An array could be initialized using curly braces or as in C or using curly braces. Accessing array's cells is done using C's subscript notation.

Like other non-static local variables, arrays are not implicitly initialized to zero.

Watch out not to give array more initializers than it already has.

An array with an empty initializer list will be set to 0 if it's type is int.

In the case of const arrays naturally they MUST be initialized.

The subscript notation in C could easily be used for C++'s STL arrays.

The array class comes with a good bunch of member functions notably:

-.size(): which returns the number of type size_t.

Btw

++arrayname[subscript]

Increments the value of subscript.

In arrays in C++ the Copy constructors are involved upon reassignments and all.

std::vector

Collections of data of same type that can grow. Each array knows its own size, which can be determined by calling its size member function.

Chapter 3: The Java Programming Language

General flow of Java programs

After the installation of Java runtime and all… and setting the PATH variables so that java compiler and java virtual machine become accessible from the commandline, we could use:

javac filename.java
to compile our java file to bytecode and then use

java filename
to run the now byte code version of the program on JVM.

Java sees every program as a bunch of classes, member functions (called methods) and their variables. Naturally as a result of this, the concept of **Global Scope**

Functions simply does not exist in Java, as all functions are **member functions/methods.**

By default Java imports the *java.lang* package which is rt.jar in its *Java/JDK/jre/lib* directory.

The content of java API's libs */rt.jars* within IDEs is usually shown as actual source code which is in contrast with how JAR files contain the bytecode, this is because for convenience Java IDE's (at least in the case of IntelliJ) view a different archive, the source code archive for the existing byte codes, which is typically in the root folder of Java and is named *src.zip*.

In short the main difference between C/C++ and java is that Java looks at source code like a bunch of already existing modules to connect are not instead of pieces that may or may not conditionally get copied into the code or not. Preprocessing doesn't much exist in Java.

Projects and package organization

Every java project must settle a directory as it's root. The files that are **just within** (NOT the subfolders or files within the subfolders) root will not belong to any packages hence should **NOT** have a settled package declaration to mark them being in the root. The files within subfolders within the root however always have to declare their path in relativism to the root.

Java Modularization

Java treats each source files as a "representative" (and home) of a public class for convenience. Naturally as a result this means each source file would get 1 public class.

1- ***Preprocessor Level (Macros)***
2- ***Source Code Level (Encapsulation)***
3- ***Compilation Level (Translation units)***
4- ***Link Level***
5- ***Build Level (libs and Usage of smaller exe s thru Command lines)***

Preprocessor Level

Source Code Level

Class

It's pretty much like in C++ except this time the access to classes can be limited. Since in Java modularization on all levels happens and gets enforced thru based classes, it'll affect other levels as well, hence, it'll show up in the latter levels.

Getters and Setters

In IntelliJ: We can use alt+Enter while on the variable line to define getter and setter for the variable

We can also use Alt+insert to add getter and setter for variables.

Abstract Class

Interface

Compilation Level

Class

In Java each a source file is only allowed to be as the placeholder of a main class or really a representative of a class with the same name inside it. Other than that there will be a compilation error. (This way of seeing things might seem a little forced and odd at first but you'll see some of the opportunities it brings such as "**fully qualified class name**" thing we've discussed in *The Language/expressions* part.

These main classes could be defined in 3/4 ways:

- Public: In which case ALL other modules could access them.

- Private: In which case only the modules within the same package could access them.

- Protected:

Packages

Packages are mere directories containing java classes. The C files defined in them must reference themselves as parts of the package using a **package declaration** at their very beginning, using relative path.

Link Level

Packages

In Java a group of classes form this concept known as packages.

JAR files

A jar file is a ZIP archive (Yes just like the same kinda zip archive you normally see in games/etc) that contains byte-code/.class files of libraries.

The Programming Process

The Entry Point

Naturally there must be an entry point for the program this entry point will be in the form of a static function (So that there wouldn't need to be an instantiation from it's class to be accessible) named main. There should be only ONE function with this name that the JVM binds to while loading .class files.

static void main (string[] args){}

is the natural entry point of java programs.

Tip: arrayname.fori is a good shortcut to create loop for array.

Methods

Even though fully possible, DO NOT call methods from within constructors. The reason has something to do with polymorphism that I'll explain in future.

The Language

Garbage Collection

When there are no more references to an object, the object is eligible to be collected. Collection typically occurs when the JVM ex ecutes its garbage collector, which may not happen for a while, or even at all before a pro gram terminates.

Keywords

Preprocessors

Import

Used to include definitions of classes.

Operators

+

Aside from it's already existing mathematical functionality that it had in C/C++, the operator allows for "seamless" concatenation of numbers/non character data into strings.

=

The assignment operator.

Comments

// and /* are the goto keywords for comments with the exact same meaning they had in C/C++.

With the addition of /** which is a more specific and preferred commenting format that causes JDK to use them in creating an HTML documentation.

Control Statements

Switch

It works as in C/C++ but now can accept non enum values such as

Loops

Range Based for Loops

Literal C++ ranged loops but it's logically impossible to change the counter elements in them since in Java much unlike C++ there's no reference-access type which is EXACTLY what you need in such scenarios.

this

The keyword is used with . to refer to current namespace as in (this.) much unlike in C++ where it was used as it->, this time it's a dereferenced-by-default pointer.

The essence of Data

The essence of data in Java just like in C/C++. Literals,vars and const vars.

Variables

Declare variables as close to where they're first used as possible.

final

Makes the variable a constant (basically const in C/C++).

Data Types

Primitive data types are written in small form while the non primitive types are written with the first letter capital.

Use camel case for variables and method and pascal case for classes.

The main difference between primitive types and user defined types Is that user defined types do NOT get their object created by default.

Java essentially says: Primitives are mine to instantiate and custom ones are yours.

All variable and object operations essentially use pointers at lower level but in Java the primitive types are made to be the direct interface for changing the value of their object (which is created and handled by default), while the reference types are to provide the indirect abstracted interface for accessing existing objects.

Basic Data Types/Primitive Types

Their size remains the same on all machinery. A primitive-type variable can hold exactly one value of its declared type at a time. For example, an int variable can store one integer at a time. When another value is assigned to that variable, the new value replaces the previous one—which is lost. **Every primitive value and object in Java can be represented as a String, they get turned to strings either explicitly(such as in concatenations) or implicitly using a toString method that returns a String representation of the object, which all the primitive types have. While the primitive values used in String concatenation are converted to Strings, the boolean ones are converted to the String "true" or "false".**

int

float

Represent single-precision floating-point num bers and can hold up to seven significant digits.

double

Represent double precision floating-point numbers. **If there are any trailing zeros in a double value, these will be discarded when the number is con verted to a String.**

Instantiation

Reference Types

Programs use variables of reference types (normally called references) to store the addresses of objects in the computer's memory. Such a variable is said to refer to an object in the program. Objects that are referenced may each contain many instance variables.

Instantiation of the non primitive types in Java doesn't work like in C++ but thru an explicit new declaration and the use of "container" (automatically dereferenced non constant pointers to data). Which is why instead of instatiating classes as we did in C++, like any other data, in Java we rely on that **new** to begin with. Such "**containers**" or **reference** types as Java likes to call them are initialized to NULL by default.

THE BEST SPECIFICALLY POSSIBLE WITH REFERENCE TYPES MOMENT FOR THE REFERENCES (THEIR SHINING MOMENT) IS IN RANGE BASED LOOPS.

Arrays (Hardcoded Reference Type)

Have to have a fixed size and are actually reference types in Java.

For arrays you can use the usual reference type initialization of using things or you can initialize them like in C with brackets leading to the virtual machine compiler's semantic analyzer to implicitly handle that. In Java arrays are a hardcoded type with extra capabilities and their own syntax. They come with a .length property that's constantly recording the length of their property.

***Arrayname*.length** is replaced with the length of the array in question.

Multi-dimensional arrays

Java due to it's further reliance on **new** for actual instantiation, doesn't provide support for multidimensional allocation for arrays.

Much unlike the C style arrays the Java multidimensional arrays are not forcibly contigious. Java sees each dimension of a multidimensional array as a reference.

ArrayLists

Could have dynamic size.

Derived Data Types

Classes

Classes are user defined types that could and usually do include a bunch of variable/ constants and some behaviors called methods.

Every class in Java has the methods of class Object (package java.lang) one of which is method finalize. You should never use method finalize, because it can cause many problems and there's uncertainty as to whether it will ever get called before a program terminates. The original intent of finalize was to allow the garbage collector to perform termi nation housekeeping on an object just before reclaiming the object's memory. Now, it's considered better practice for any class that uses system resources—such as files on disk— to provide a method that programmers can call to release resources when they're no longer needed in a program. AutoClosable objects reduce the likelihood of resource leaks when you use them with the try-with-resources statement. As its name implies, an AutoClos able object is closed automatically, once a try-with-resources statement finishes using the object.

Unlike in C++ where classes themselves did not have access modifiers and were always globally available, in Java the availability of the classes is defined by their access modifier and there wont be a semicolon after the definition. Each class definition follows the following format:

AccessModifier *class* **className** *{}*

Due to the intertwined nature of classes and files in aside from what was already said any public classes must be declared in a file with the same name. If no access modifier is written for the fields or methods of a class they'll have package access

by default. Classes in the same source file are considered as part of the same package.

Having multiple classes in one source file without defining access modifier leads to java compiler making multiple class files as a result.

Fields

Unlike in C++ instance variables access modifiers do not use the colon format but are simply prefixed to the instance variables, per variable.

The format is :

AccessModifier Type name;

Conventionally we put the instance variables on the top of the class so that it could be seen first.

Note: Fields ARE initialized by a default initial value.

this

The keyword **this** has the exact meaning as it did in C++. Since in Java we don't have pointers, the this means the **dereferenced** pointer to the object not the mere pointer. So, in Java:

this == ***this**(in C++)

Class Variables / Static Variables

These variables are made only once. Not per instantiation.

Class's Instantiation

In Java the instantiation of classes is an unusually extra stepped declaration in comparison to C++ as instead of using the format:

Type name *;*

That was used in C++ and for Java's normal variables, you have to deliberately assign the named declaration to an object of it using keyword *new*:

Type name = new Type() *;*

new

The keyword new allocate memory for the obj it seems / Creates object of the specified class.

By default the constructor of the class is called upon it's instantiation possibly asking for arguments according to its definition.

Subclass

Subclasses are classes that are inherited from the other one.Constractors can also be inherited by subclasses but their name sensibly requires a change. That's where a super constractor call comes in handy. A super call basically calls the parent constractor within the constractor of the child class.

Abstract Class

An abstract class contains prototypes for functions to be defined in the subclasses. When trying to create an object of an abstract class the abstract methods must be defined right after the constractor in a block as well. we use implements to inherit abstract classes

Interface

Interfaces are just abstract classes that are only prototypes. In interfaces access modifiers are public by default. Interface reasonably can't inherit (extend) a class but only extends an interface

Enum

An enum type is a special user-defined type declares a set of constants represented by identifiers:

AccessModifier enum EnumTypeName { CONSTANT1,CONSTANT2,…};

An instance of enum type can hold one of it’s members at a time. In order to use enum types we first make an instance of their type and then proceed to initialize, assign that instance using:

EnumTypeName.MemberName

notion.

Java does not allow an int to be compared to an enum constant.

An enum class has three parts:

- enum constants showing up a objects of its own kind that possibly receive inputs as values for their constructor in the format:
CONSTANT1(),CONSTANT2(),CONSTANT3();

- Constant instance variables to be set
- methods

It’s safe to say enums are constructed as classes composed of themselves.

In an enum declaration, it’s a syntax error to declare enum constants after the enum type’s constructors, fields and methods.

Generic Methods

Used when types are different but the method's action is the same. First we hage to define a type parameter like this. right after name

Generic Methods

Instantiation: ClassName objname= new classname<>();

Passed by Value vs Reference

The values in Java are passed by value by default but for classes and all since the reference and the copy of it remain to the same pointer ultimately, they point at the same object.

Methods

In Java methods are defined once in the java class with the access modifiers written per function. The rest of the format is identical to the one in C/C++:

AccessModifier returnType className (1st ParamType 1st ParamName,2nd ParamType 2nd ParamName).

Note: Local variables are NOT auto initialized by default.

The methods regardless of their static ness are accessed using the . member

notation.

Static Methods

A static import declaration enables you to import the static members of a class or interface so you can access them via their unqualified names in your class—that is, the class name and a dot (.) are not required when using an imported static member.

Static methods could be imported as single methods or as a collection of the static methods of a class using the following import templates:

import static packageName.ClassName.staticMemberName;

OR

import static packageName.ClassName.*;

Variable Length Argument Lists

The format for methods that have variable length arguments is

returnType FunctionName (type param1,type 2 param2,type …NameOfArrayOfVariableLengthArguments)

In java it actually creates an array of those parameters with the name of the parameter as the array name. These args are then typically accessed using ranged for loop.

Local Variables of Method

Local variables of a method are NOT initialized by default.
Attempting to use the un initialized local variable results in a compilation error.

Method Overload

In order for methods to be overloaded we have to have atleast an extra parameter in the similar method or a parameter with a different type or sequence.The return type does not matter.

Method Override

Overriding== We have a method that takes exactly the same arguments(calls the first method in it's body) + extra things in it's own body. Overriding methods can be achieved by using the method prototype and then defining the new body in it's overriding method in a subclass. The return type name and parameters have to be the same in overriding. We also can't have more limited access in the overriding methods only less limited access modifiers are allowed as our overriding methods require at least the same level of freedom as the overrided method.Static Methods can't be overrided.

Final Methods

Final type methods cannot be overrided.

Constructors

A constractor is a method with the name of its class that makes sure the variables that are in its parameter list are innitialized. It MUST and WILL be defined, only either implicitly by the compiler (in which case it'd be called a **Default Constructor)** or explicitly by the user as it has to be called to be run for the object of a class upon its instantiation.

Constructors CANNOT return values.

Constructor Delegation

Due to the absence of initializer lists for constructors the constructor delegation happens in the body of constructors in Java and with the keyword **this** replacing the name of the constructor. The constructor delegation must be the first statement within the constructor.

Type Conversion Methods

Exceptions

Exception handling allows for the creation of fault-tolerant programs. When a method or JVM encounters/detects a problem, it throws an exception.

try{}

We place the codes that might throw exception in the flow of program in a try block. Causing the compiler to keep that chunk/range/section of code in mind and jump to the following catch chunk upon encountering an error within that range.

catch(){}

The catch block is regarded as the immediate solution block for possible errors encountered in the try blocks. It's run only and only if the try block actually throws an exception.

throw

It throws a type as as exception.

Access Modifiers

Public

Allows accessibility from any other scope.

Private

Allows accessibility only within the scope.

Storage Classes

Non Static

In class scope

Variables

These variables are called Instance variables. Each instance of the class gets it’s own copy of them.

Methods

Static

In class scope

Variables

These variables are known as **class/static variables**.

Methods

Such methods are referred to as **class/static/instance methods**.

Expressions

Fully Qualified Class Name

It allows you to use Java's relative addressing system to load the class without importing it.

The Java API / Standard Library

The Java's API is usually (in IDEs like IntelliJ at least) in visible under the External Libraries section in solution explorer section of the IDE. It is implemented as a set of JAR files(ZIP format Archives) containing a set of directories and the metadata required for unarchiving them correctly. As a result these JAR files could contain from "folders" to files/public class containers.

With the IDE's usually importing Java API by default as they also set the directory of their residence set an include directory, there aint much need for importing them.

By default (IntelliJ IDE at least) has the root of import set at the **rt.jar**.

String (class)

The type allows for concatenation using +. As it did in C++ and functions similarly. What's funny is that the normal string literal concatenation we had in C and C++ actually DOES NOT work here and we always have to use + to achieve concatenation regardless of the kind of the operands.

System (class)

Contains stuff… related to System. Prominently used for I/O operations.

E.g: System.out.println("String").

Contains:

- an object of type PrintStream named **out** thru composition.

- an object of type InputStream named **in** thru composition.

The static methods:

- arraycopy()

PrintStream (class)

Provide options for printing. Some known methods involve:

- print()

- println()

- printf()

Scanner (class)

Defined within rt.jar.java.util.Scanner it is used as a place holder object that could be tuned to accept inputs. Contains the following methods:

- Scanner(filestream): Constructor used to set where to read the input from.

- nextInt(): Reads another int from the Istream, if it's empty it asks for input using the terminal.

- nextLine(): Reads characters until it reaches newline/Enter character which it discards and replaces with \0.

- hasNext(): The function checks to see if the EOF has been reached, if the buffer is not EOF and not empty it returns true, if the buffer's not EOF and empty it gets a token.

SecureRandom (class)

Defined within rt.jar.java.security.SecureRandom

- nextInt(): Leads to random number generation, within the 0<given input range if given any.

Integer (class)

Has members notably:

- parseInt: Turns a string of integer to its integer value, in one rack.

ArrayIndexOutOfBoundsException (class)

Type of exception thrown upon array going out of bounds.

Arrays(class)

Located in java.util.Arrays

Has static members:

- binarySearch:

- sort:

- arraycopy:

- fill:

- equals:

Integer(class)

It has the static methods:

- parseInt():

BigDecimal(class)

Located in java.math:

NumberFormat(class)

Located int java.text

Java Collection Classes

These classes provide efficient methods that organize, store and retrieve your data without requiring knowledge of how the data is being stored. This reduces application-development time.

ArrayList (Generic Class)

Located in the java.util

ArrayList <typename> name;

You cant use primitive types to declare variables and create objects of generic classes.

It contains:

Static Methods:

- add:
- clear:
- contains:
- get:
- indexOf:
- remove:
- size:
- trimToSize:

Java FX Library

Chapter 4: The x86 Assembly

MASM

Chapter 5: The XML

The x86 assembly actually has 2 syntaxes one is MASM

AT&T

Chapter The Python Programming Language

Environment & Tools

Industry hack: The python code is used in small chunks and run thru bash/batch files indirectly for checking and debugging different parts of the apps.

File Management

Dynamic Libraries

CRT is a block o code that's added before the main function and is responsible for calculating and "saving" command line arguments and possibly assigning the main's parameters by them according to the standards(such as the case with argc and argv).

Textures

Texture Compression

BC7

Texture mapping

My basic analysis

Witcher 2 executable

Chapter 1: Developing an OS

"To write an OS for a Von Neumann computer, a programmer needs to be able to understand and write code that controls the cores components: CPU, memory, I/O devices, and bus."__Operating Systems from 0 to 1

Subject 3: Operating Systems

OS Concepts

(System's Engineering, Systems Programming,System's Design Level)In this layer you deal with computer organization. Firmwares, BIOS and etc.

Memory

Logically memory is organized as a linear array of bytes, each with its own unique address (array index) starting at zero. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Programs use different sizes of memory and to be able to have control over the bytes of memory, they NEED to have unique identifiers for each of those bytes. With data representation in computers ultimately happening by bits, the same unit is used to state the length of the those unique identifiers for memory bytes in computers, the length is referred to as **address space** and has direct effect on how much memory you can use at the same time.

Memory Hierarchy

The main idea behind a memory hierarchy is that storage at one level serves as a cache for storage at the next lower level.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Direct Memory Access

DMA

Virtual Memory Address (VMA)

Why it's a thing: The programs can't know for certain in which memory addresses they're gonna be loaded until their runtime but until then they still gotta rely on some independent in house addressing system to do their relocations and all and that's how and why they use the virtual memory address space which is a middle

man addressing system keeping the program connected until it's actually loaded at runtime. Now the number of unique memory addresses or to be more frank, the size of memory that can be accessed at the same time, is define by how many bits long the address space can be.

It's safe to say virtual memory is an abstracted interface to access dynamic random access memory (DRAM), flash memory, disk storage and special hardware.

A machine level program views memory as a very large array of bytes referred to as virtual memory. Each byte of which is represented by a unique number, called a virtual address. The set of virtual addresses is referred to as virtual address space. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Word size

Every computer has a word size defines the nominal size of pointer data, the range of Virtual address is one of the many things determined by this word size. COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Memory Page

Why is it even a thing: To manage apps that areNow it is neither convenient nor optimized really to load apps at their entirety, hell the memory of the computer may NOT EVEN BE ENOUGH to hold them entirely. Each of these chunks have a contagious group of addresses is called a **page**. The pages of an app form what's **virtual address space**. For loading the app the wrapper unit for determining how much **virtual** memory is used is **page** (which itself is based on bytes obviously) and each page is a contagious series of bytes. For physical memory the same kinda wrapper unit exists and is called **pageframe**. The pages and page frames typically are of the same size.

System Call

Is a special function that passes control to the operating system.

COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE

Context Switches are managed thru system calls to kernel for managing process creation.

Kernel

Kernel is part of the operating system that always resides in memory. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Kernel is not a separate process but a collection of code and data structures that system uses to manage all processes. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

CPUs

A central processing unit is typically made of one or more processing cores, cores each capable of handling one sequence of instructions at a time, on their own.

Process

A process is the operating systems· abstraction for a running program. Multiple processes can run concurrently on the same system, and each process appears to have exclusive use of the

hardware. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Context Switching

Traditional systems could only execute one program at a time, while newer multi core processors can execute several programs simultaneously. In either case, a single CPU can appear to execute multiple processes concurrently by having the processor switch among them. Tue operating system performs this interleaving with a mechanism known as context switching. When the operating system decides to transfer control from the current process to some new process, it performs a context switch by saving the context of the current process, restoring the context of the new process, and then passing control to the new process. The new process picks up exactly where it left off. **COMPUTER SYSTEMS A PROGRAMMER'S PERSPECTIVE**

Now obviously the number of tasks AKA sequences of instructions we are running may exceed the number of cores which is why these cores use a technique known as context switching to “Jump” on another

Threads

A CPU/Processing core executes lines/threads of instructions by reading them thru it’s own threads. Each processor and application can consist of many threads executing within it’s address space, which the processor’s threads could lock onto. The threshold of the number of threads that a processor can simultaneously execute is naturally limited to the number of threads it has.

Multi-Threading

In case a CPUs total threads are lower than the number of threads in a process, context switching becomes the go-to solution. Otherwise CPU threads each get assigned to a process thread and handle them simultaneously

In case of hyperthreading, each core technically achieves the illusion of handling multiple threads thru its

The virtual memory of a process contains different sections for code, data, heap , shared libs, stack and kernel virtual memory.

Multi-Threading

A process can actually consist•of multiple execution units, called threads, each running in the context of the process "11-d sharing the same code and global data'.

Graphics APIs

Chapter 1:Windows

Process

Is an OS managed container/runtime environment for running a program.

Kernel

The kernel is more like a piece of software with other modules/ kernel objects.

ICD(s)

Windows manages the components of computer thru and using their drivers also referred to as **Installable Client Drivers/ICD s**. The location of these ICDs is typically kept at the windows' registry/registry editor. These ICDs are usually implemented as DLLs, residing in conventional locations. The ICDs also follow a specific ABI to make them independent from import libraries. For different drivers Windows expects them to be obeying different ABIs.

OpenGL

In Windows OpenGL is implemented as an OpenGL32.dll, the functions of which get the adapter info and load the appropriate ICD DLLs. The way windows does load it is thru runtime loading within the OpenGL's runtime and subsystem.

Registry

Includes locations of **ICDs ()**.

Loader

Checks the PE files to load their DLLs according to the DLL loading rules. The locations of used DLLs in an app is kept in the import Directory entry of the PE file

ABI(s)

For 32 bit applications for x86 processors Windows demands that applications or callee s handle their stack operations and account for it.

Module(s)

Program(s) loaded and sitting in the virtual memory of the computer.

Window Class Registry

The USER32 subsystem has a class table in which it holds the template of class for later-reuse in an efficient manner.

Dynamic Link Libraries (DLLs)

Dynamic Link Libraries are loaded in two ways in Windows:

- Through an import library

- At runtime

Typically and usually the first approach is more common.

Import Library Case

In this approach a static lib is used and injected within the executable containing the address of functions and the DLL name in the PE. Windows follows a search order finding the DLL, within a hierarchy. The import library is a static lib

generated for the DLL specifically. How it's done is a matter of Windows API again.

At runtime

We'll get to the second one in the Windows API chapter as it happens using a windows function.

Programming

Chapter 0: Notable APIs

Chapter 1: Windows Programming (WINAPI)

Environment & Tools

In order to develop applications using this API having/downloading the Microsoft Windows development tools (headers, compiler and all) is necessary. Windows greatly depends on 2 main Dynamic Libraries; one of which includes Kernel mode side of the functions with the other one including the User mode side functions to achieve many of it's tasks. With the User mode dynamic library containing the functionality and being the most bare minimum abstraction that the Win programmers usually need, the header containing the function declarations of it are packed alongside the static reference libs, Microsoft's visual C/C++ compiler, a text editor, debugger and many much helpful necessary tools to be given as visual studio and it's windows SDK to the users.

The actual implementation of Windows API resides in User32.dll which is in

YOURWINDOWSINSTALLATIONDRIVE\Window\System32 (for 64-bit Systems)

YOURWINDOWSINSTALLATIONDRIVE\Window\SyWOW64 (for 32-bit Systems)

While the static reference libs for linking to these dlls are provided as a part of the Windows SDK.

_WIN32 is a preprocessor used for conditional inclusion of Windows specific code (for both 64 and 32 bit compilation).

_WIN 64 is for 64 bit Windows .

General Flow of Windows API Programming

The process of windows programming always involves the act of defining the entry point, having the handles set,

Entry Point

The WINMAIN gets the base address (the address at which the executable is loaded into memory) as hInstance at runtime, next the single string of command line argument string (NOT like in standard C's argv which is an array of strings that C runtime has already taken care of by separating arguments based on whitespace by default) is recieved thru cmd as szCmdLine and the default state in which process is run based on OS's assessments is received iCmdShow to then be

passed as a parameter to our ShowWindow() function. Regard iCmdShow pretty much as a bitfield of window properties to later use for showing window.

DLLs

As mentioned previously in the ***Operating Systems*** the loading process for DLLs could be done in two ways:

1- Through Import Libraries
2- At Runtime

Through Import Libraries

The executable file will contain the DLL name and function addresses in **it's own section within the PE format of the executable**.

At Runtime

Using the **LoadLibraryW()** and **GetProocAddress()** functions **the code section of our program** addressed the functions and links them.

Approaches

WinAPI specifics

“Back-end”

ABI Conventions

For 32 bit x86 Windows programs Windows demands you stick to and follow the WINAPI calling conventions (such as CALLBACK and WINAPI) that causes the generated binary to tell windows in explicit that it will take care of the cleaning process of its own stack.

These ABI conventions came to be so that Windows and Apps could have an understanding of their responsibilities in a less ambiguous manner.

It will know “before hand” that “ okay this program will take care of its stack, not my business to do the cleaning and possibly destroy it unwantedly”.

Window Class

A window class acts like a template for windows to later create a window from. It DOESN’T MEAN the programming language level abstraction, class and serves more like a custom place holder memory chunk containing the template info for your windows.

C/C++

C#

“Front End”

Window(s)

A Window is a generic structure. In order to create one we first create an instance of this structure. Then we MUST configure it according to our ideal so that when we’re done we can register it for the backend side.

Windows programs typically rely on communicating with “Window” objects using concepts known as **messages**. These **messages** are handled by a role known as a “**window procedure**”, a function the parameters of which describe the particular message that is being sent by Windows and received by program.

Every Window has an associated window procedure which could be in the program or even a dynamic lib that the windows uses by sending messages to it’s window procedure.

The process of it’s creation involves:

1- **Defining and setting the structure of window class**
2- **Registering the Window Class**
3- **Creating from that Registered class**
4- **Showing the created WNDClass**
5- **Handling the messages**

1. Defining and setting the structure of Window Class

Create an object of type (struct type) WNDCLASS and set it's attributes.
A window class has a number of properties, some mandatory to initialize:

1-

2- Window Procedure:

2. Registering the Window Class

The class will be registered using the RegisterClass function.

3. Creating from that Registered Class

The registered class will then be instantiated using the CreateWindow function.

4. Showing the created WNDClass

The created instance of Registered Class is then showed using ShowWindow.

The Message Queue

For all windows programs a queue is made, holding all the messages to all the windows a program might create

Window Pump / Window Message Loop

In order to read and manage the messages of the queue according to the window of the application, Window pump/Message loop s are developed.

It's typically the main loop of the window and your entire code for that must be implemented in the body of it.

```
while(GetMessage(****))

{

\\Your code

}
```

Window Procedure

Each window in it's structure has a role known as a **window procedure** perfectly carved in the form of a pointer to function that is to decide the way the window is to handle everything. Hence upon defining a function of the format that matches the pointer's desired pointee type, we just give the name(address) of the function to the pointer.

```
Wndclass wnd;

wnd.lpfnWndProc = WndProc;
```

Handle(s)

A handle in Windows is an identifier that windows gives to program for something that it's managing.

C/C++

Internationalization

Developing International Software for Windows 95 and Windows NT by Nadine Kano.

The Usermode DLL

Before the 32 bit era there was the 16 bit era in which the DLL for user mode side functions was a file called USER.EXE (Odd right but seriously it functioned as a DLL back then). It wasn't until the 32 bit versions of Windows showed up that the User

The implementation of much of Windows's types and MSVC compiler's behavior is typically achieved through:

1- **The CSFOA method**: You see, considering how much of Windows was first written in C before C++ and function overloading was a thing programmers had no option BUT to always define the "technically" overload versions of a function with different names and to give the illusion of overloading the solution was to use conditional preprocessor
2- **SAL**: Windows API uses a MACRO convention style known as SAL for it's implementation.

SAL

Is a descriptive language for defining macros used by Microsoft Windows almost everywhere for the implementation of Windows API and is thoroughly covered in sal.h header that comes as a part of the SDK

AFX Resource Language/Script

Running Executables with parameters

By dragging files onto executables the address of them is given to executables(the directory of them/file.extension is given to the executables as parameter).

Resource Files

Resource files are files deployed alongside applications that fill gaps such as icons and etc.

.ico

An .ico file is a container for a number of image files to be used later, this container typically involves:

1- **A 32-bit 256x256px PNG image**: This first high res image is typically compressed in PNG format as having it as an uncompressed horrendeous sized bit map is not that sensible.
2- **An 8-bit 48x48px bitmap.**
3- **An 8-bit 32x32px bitmap.**
4- **An 8-bit 16x16px bitmap.**

These files typically contain a PNG and a couple of bitmaps. A big quality 256x256 pixel image followed by 8 bit bitmap images. The reason is that having a compressed PNG is better than having an uncompressed bit-map for a large 256x256 pixeled image!

The smaller files

Resource IDs

Now these IDs are used to reference the existing resources. The size of the data type used for them could defer but they use INTENTIONAL truncations based on decided datatypes to achieve desired results.

Constants

CS_HREDRAW

Sets wndclass.style for redraw upon every change in height.

CS_VREDRAW

Sets wndclass.style for redraw upon every change in width.

MAX_PATH (minwindef.h)

Is a Windows SDK constant to set buffer for directory of setActiveDirectory and all.

Preprocessors

_Unicode

Defined in UNICODE, it’s used to include the Unicode version of tchar functions.

TEXT(quote)

Turns string literal into the ASCII/Unicode syntax for initialization using the __TEXT(quote) Preprocessor.

__TEXT(quote)

It's a conditional preprocessor loading the string literal quotes according to the syntax.

DECLARE_HANDLE(NAME)

Used for the creation of handle types, defines NAME as a "pointer to a NAME_ struct" type with a single int member.

Entry Point

The entry point of the average WinAPI program instead of the good old:

```
int main (int argc,char*args[]){/*program*/}
```

in C/C++ is the *WinMain* function with fixed parameters that tend be defined using the naming convention of one of the two prototypes below:

```
int  WINAPI WinMain  (HINSTANCE  hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int  nShowCmd)
int  WINAPI WinMain  (HINSTANCE  hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int  iShowCmd)
```

where function's return type is *int WINAPI* (*WINAPI* being a convention for identifying the return type of WinAPI functions, it is mostly used in them).

- The *first-parameter (hInstance* here*)* is an instance handle, the unique number that identifies the program.
- The *second-parameter* (*hPrevInstance* here*)* was used to determine if other instances of program were running and now its abandoned in 32 bit Windows and is always defined as NULL in modern Windows.

- The *third-parameter (szCmdLine* here*)* is the command line used to run the program (some apps use to load a file into memory when the program is started), the **sz** here stands for zero terminated string.
- The *fourth-parameter* (*iShowCmd* here*)* is used to set whether the program should initially fill the window or be minimized or whatnot the i stands for integer in the name.

Functions

WinMain()

The prototype is typically one of the lower:

```
int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd)
int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int iShowCmd)
```

MessageBox()

A CSFOA used to show short messages, it shows a dialogbox and it ain't much customizable. The prototype is:

```
int WINAPI MessageBox ( HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType);
```

- The *first-parameter (hWnd* here*)* is supposed to get the handle of the possible "**owner**" or "**parent**" of it (**OR NULL if an owner/parent doesn't exist**) to process the message box relative to it.

- The *second-parameter (lpText* here*)* is supposed to be the text of the **body**.

- The *third-parameter (lpCaption* here*)* is supposed to be the text of **title** of MessageBox.

- The *fourth-parameter (uType* here) is a constant (possibly a bitwise combination of multiple constants) for setting flags to add some predefined and limited icons and options to the window. Using NULL or 0 here will give the default window with the default buttons,the default icon and the default selected-by-default button.

The full list of the constants will be written in Apendix A.

The message box tends to return different values based on buttons present varying from IDYES,IDNO,IDCANCEL,IDABORT,IDRETRY,IDIGNORE to 0 and 1.

When using this function we tend to put the string literal parts within a TEXT macro which is optional but helps in case we wanted to be ready to convert our programs to Unicode character set.

For now we use build ctrl + F7 to build project and then ctrl + F5 to run it.

wcslen()

Is used to determine the length of widechar array minus the '\0' character and is defined in both string.h and wchar.h.

wprintf()

Defined in both standard C's stdio.h, AND wchar.h.

lstrlen()

CSFOA used to calculate the length of string minus the terminating '\0' char.

lstrcpy()

CSFOA used to copy the second string into the first.

lstrcpyn()

CSFOA used to copy the first 3rd parameter's number of elements to the first argument.

lstrcat()

CSFOA used to concatenate the first argument with the second one.

lstrcmp()

CSFOA Compares two string char by char if all are equal(in size and value, meaning strings are totally identical) returns 0.

lstrcmpi()

CSFOA does lstrcmp but for the number of characters given in the third parameter.

sprintf() (C stdlib)

```
int  sprintf  ( char  * szBuffer, const char *szFormat, ...);
```

The function is used to hold formatted string in a buffer.

vsprintf()

A variation of sprintf, where the first two arguments are char buffer and format string but the third is actually an array of arguments instead of being … .

GetSystemMetrics()

A useful function for obtaining info about the sizes of various objects in windows.

LoadIcon()

Used to load an icon for use by a program. First argument is the hinstance of EXE in which to look for and the second one is the ID of the

LoadCursor()

Used to load a mouse cursor for use by a program.

GetStockObject()

Gets a graphic object.

RegisterClass()

Actually registers class within the user32 subsystem as a template (read **Window Class Registry**).

CreateWindow(DWORD)

CSFOA for creating window, can possibly expand to:

```
1- CreateWindowA(lpClassName, lpWindowName, dwStyle, x, y,\
nWidth, nHeight, hWndParent, hMenu, hInstance, lpParam)\
CreateWindowExA(0L, lpClassName, lpWindowName, dwStyle, x, y,\
nWidth, nHeight, hWndParent, hMenu, hInstance, lpParam)

2- CreateWindowW(lpClassName, lpWindowName, dwStyle, x, y,\
nWidth, nHeight, hWndParent, hMenu, hInstance, lpParam)\
CreateWindowExW(0L, lpClassName, lpWindowName, dwStyle, x, y,\
nWidth, nHeight, hWndParent, hMenu, hInstance, lpParam)
```

PCTSTR lpClassName: Is the registered Window Class.

PCTSTR lpWindowName : Window Title bar.

DWORD dwStyle: Is style for Window.

int x: the x position.

int y: the y Position.

int nWidth: width.

int nHeight: height.

HWND hWndParent: A handle to the parent or owner window of the window being created. To create a child window or an owned window, supply a valid window handle. This parameter is optional for pop-up windows. To create a message-only window, supply HWND_MESSAGE or a handle to an existing message-only window.

HMENU hMenu :A handle to a menu, or specifies a child-window identifier depending on the window style. For an overlapped or pop-up window, hMenu identifies the menu to be used with the window; it can be NULL if the class menu is to be used. For a child window, hMenu specifies the child window identifier, an integer value used by a dialog box control to notify its parent about events. The

application determines the child-window identifier; it must be unique for all child windows with the same parent window.

ShowWindow()

Sets the window for showing.

UpdateWindow ()

Renders the window according to the latest changes.

GetMessage()

Acts like C’s getchar() but for a Window. Returns 0 if user

UpdateWindow ()

Directs the window to paint itself.

TranslateMessage()

Translates some keyboard messages.

DispatchMessage()

Sends a message to a window procedure.

PlaySound()

Plays a sound file.

BeginPaint()

Initiates the beginning of window painting.

MAKEINTRESOURCE()

SetPixelFormat()

HGLRC wglCreateContext(HDC unnamedParam1)

HGLRC wglCreateContext(HDC unnamedParam1)

Creates a new OpenGL rendering context.

GetModuleFileName()

Located in libloaderapi.h , the function returns a null terminated string literal (char*) path of the directory of the module belonging to the current process.

Headers

vcruntime.h

Much of the fundamental typedefs and build dependent size determination for implementing C and C++ is done in this header file using macros.

wchar.h

Contains the type definitions for wchar_t data type and wide char related functions such as wcslen.

string.h

Contains string related stuff and functions such as strlen.

windef.h

Includes type definitions and certain functions for them.

tchar.h

Acts as a cross-encoding supporting (ASCII & UNICODE) header abstraction for functions.

winnt.h

This bad boy handles the basic Unicode support.

Types

wchar_t (C stdlib)

2byte charcters for representing Unicode. Literals of this type are to be prefixed by L (without any space in between) to indicate that each character in the literal is to represent 2 bytes.

TCHAR (Windows Generic/Universal Char winnt.h)

Is a generic character type used determined at compile time based on whether _UNICODE has been specified or not.

CHAR (Windows 8bit char winnt.h)

Window's middle type name for 8 bit character.

WCHAR(Windows 16bit char winnt.h)

Window's middle type name for 16 bit character.

PWCHAR, LPWCH, PWCH, NWPSTR,LPWSTR, PWSTR (Windows defined in winnt.h)

All typedefs for pointer to WCHAR (WCHAR*).

LPCWCH, PCWCH, LPCWSTER, PCWSTR (Windows winnt.h)

All typedefs for pointer to Const WCHAR.

PTCHAR

Pointer to generic/universal char.

LPTCH, PTCH, PTSTR, LPTSTR

Pointer to the genetic char type.

LPCTSTR

Pointer to constant generic char type.

WNDCLASS

A type used to keep the structure of a window. A Window Class Structure.

HWND

A handle?

MSG

Message structure

HDC

Handle to a device context. It’s the middle man context that gives info to graphical output according to the interface of choice. Defined in Windef.h

PAINTSTRUCT

Paint structure.

RECT

Rectangle structure.

PIXELFORMATDESCRIPTOR(struct) (wingdi.h)

Used to define the pixel format we wanna attach to HDC It’s actually a structure containing many variables notably, used to describe a set of properties we want the final pixel format for our HDC to support:

nsize(WORD)

The size of structure, must be calculated and initialized thru sizeof().

nVersion(WORD)

Usually 1, used to indicate the version of pixelformatdescriptor interface being used.

General acceptable value for it is 1.

DwFlags(DWORD)

Indicates the kinda abilities we want the format to have.

Possible values that could be given to it are:

- **PFD_DRAW_TO_WINDOW**: The pixel format can be used for window.

- **PFD_SUPPORT_OPENGL**: The pixel format will support OpenGL.

- **PFD_DOUBLEBUFFER**: The pixel format provides support for double buffering. Since single buffering aint no fun, the single buffer is constantly loading pixels asap, NOT full images at once, leading to flickering.

Or any mixture

HGLRC

Rendering Context

Message Pump

It’s a strategy to keep the window loop in the rendering mode.

NULL

Is defined in windows vcruntime.h.

Chapter 6: Data Compression

Data compression has to do with the methods and algorithms to believe it or not compress data (serious shit!).

Good examples of the kinda historical methods people came up with to achieve it are Braille,morse codes and vcoder(voice recorders) where:

1- (in morse code) Assigning less demanding forms to more frequently used alphabets (which was later used in Hoffman encoding as well)
2- (in braille grade II) Using a special symbol to determine whether the following content were to be read as word or alphabet, taking advantage of less symbols to cover more frequent words, leading to 20% compression.

3- (in Vcoder) the data related to frequencies that could not be heard by man were just avoided  .

Lossy/Lossless Compression

Subject 4: Data and Programs Storage and Processing

Chapter 0: General Concepts

Encodings

Format

Endianness

Metadata

Chapter 1: ELF Format

Chapter 2: PE/COFF Format

Chapter 3: ZIP Format

Chapter 4: MPEG Family

Subject 5: Computer Networks

Chapter 0: The General Flow

Introduction

The communication between numbers of computer devices with each other for sending and receiving data in the form of packets lead to the formation of computer networks (Which are protocol-ed communions between computers). The connection of computer networks to other computer networks and the exponential extension of the network lead to the formation of the **internet/interconnected network** as we know it (which is an extremely large protocol-ed communion/gathering of computers).The communion itself is made possible thanks to the implementations of certain general and purpose specific key concepts in the form of hardware AND software infrastructure that exist to specifically handle the process of taking and posting packets from and to the computers.

Section 1: The Basics

Lesson 1: Internet from An Abstracted View

Key Roles

Now for the formation of a computer network there are certain logical roles that would be necessary to fulfill different yet equally necessary aspects of the job. For easier understanding consider Internet as a massive network of Post-offices for computers that it is. There are the Writer and Target Recipient devices for packets that are both referred to as **End Systems or Hosts** (Didn't use the terms sender/receiver cause many devices might technically "send" and "receive" the packet in the process until it reaches the desired destination) and then there are the "post office" devices which receive and dispatch the packets to their destinations using their links and are called **Packet Switches**. The end systems and packet switches communicate and send packets through their **socket interfaces** and follow certain **protocols** so that the **packets** (yes the data that computers send in networks are literally called packets here and are sent in similar fashion as well !) are sent to the right destination based on a list of allowed routes.

Conclusion

The whole functionality of internet ultimately really relies on 5 logical basic concepts:

- Packets: Structured data (based on protocols/standards) ready to be sent to a single or a group of end systems.

- Protocols: The structure rules for packets, requests and etc and how they are meant to be sent.

- End Systems: The systems that post/receive packets.Servers,Phones,Computers,etc.

- Socket Interfaces: The hardware/software infrastructure responsible for sending/receiving the packets.

- Packet Switches: The devices that provide route/link to other End Systems.

Lesson 2: Connection to Internet

Packet Switches' Goal Division

For our devices to "connect to" or "access the internet", they must contact the packet switches that allow routes to the "target recipient" of their packets (otherwise they could not be delivered),as everytime we use internet we're sending some request to some device and then (usually) getting some request from that very device in response. Divided by purpose, we have 2 main groups of packet switches that are made to serve different yet equally necessary parts of the process of packet delivery:

- Link Layer Switches: They form a district of all the devices that are connected to them by providing them all with **usable links** that the connected devices can always use for sending/receiving packets to and from all other devices that are connected to the Link Layer Switch. Going with the previous post office example it's safe to say they act as the "neighborhood" local post office.

(Note: The main concern of the link layer switch is not "whether to provide a link or not" but "how to provide the link")

- Routers: They regulate the transfer of packets using their gates to make sure only the packets that meet the priorities and rules defined by the operator makes it. They are conditional connectors. (THEY ARE THE DISPATCHE).

Lesson 3: Access Networks

Knowing the purpose of Routers and Link Layer Switches, if we try to model internet we'd see how both the End Systems and the first router of a network

always end up on the sides/ends/edges of the network. End systems of a network logically and as the name suggests …"end" up being on the end/edge of the network as they are its limits and then its first router (being the connector of the said network to the other ones that it is), also ends up on the edge of it (Used the word end a little too many times there I know Lol but it's to fully install the idea in your head).That's why access to a first router (Usually called the **Edge Router**) in order to access the internet becomes an important and deciding factor. The infrastructure/kind of network that exists to connect separate end systems/computer networks to the edge router is called an **Access Network (<u>In other words an access network is:</u>**
<u>a computer network/end systems + the infrastructure that connects it to its first router</u>).Since the subject is one of those parts that could easily become extremely confusing. For simplicity and better understanding how access networks work, I use the following ring observation and approach to teach what is actually happening:

1- Ring 0: The Pre-Edge Router Network/ End Systems
2- Ring 1: Linking to the Routers

1.Ring 0: The Pre Edge Router Network/ End Systems

This is the inner-most ring we always start from. "The computer network/The end systems?! The hell is that supposed to even mean" you see in this ring we ultimately have one of the 2 scenarios mentioned below (And they are NOT THE SAME):

A- Separate End Systems : Just a bunch of end systems **without connection to each other.**

B- A Computer Network (LAN): A group/pack of end systems **that are actually connected to one another as well.**

In scenario A at the ring 0 we only have the End Systems and NOTHING more, we ourselves do not "have" our own computer network. In scenario B however we already have a small computer network of our own (also referred to as a local area network or LAN) which necessates the addition of a link layer switch and it's linking infrastructure to the ring to form that "pack" of devices(So scenario B is conceptually: Scenario A with links in between).

The linking to the link layer switch happens in one of the ways mentioned below:

1- Wireless: In this kind the link layer switch itself or some connectable infrastructure to it provides what's called a base station (AKA Access Point or AP for short) that could be used by supporting devices for wireless connection.

2- Wired: In this kind of connection, Ethernet cables are used to connect devices to the link layer switch.

2.Ring 1: Linking to the Routers

Even the edge routers could exist at extremely varying distances and the scenario we have in ring 0 is likely to indirectly affect that. For . If it's scenario A we are dealing we are dealing with. Just like with the link layer switches,there are 2 general groups of common ways to link to the edge router, that are used under different circumstances:

1- Wireless: The connection to the routers happens through a satellite like with starlink/hughesnet and etc or using an access point (just like the case with link layer switches).
2- Wired: The connection happens using physical connection. It's still more common.

For wired connections depending on how far the router is engineers came up with some damn good strategies on how to connect to the routers which as I'm writing this have remained the famous and mainstream ways so far.These 4 common types of wired links to the router from a connection infrastructure perspective are:

1- DSL: It's common in home access.In this kind of edge router link local teleco transfers data to the edge router and vice versa using the existing Telephone lines(not coaxial cables). This happens by grouping the data on the phone lines using frequency ranges(One range for two-way telephone channel,one for uploading/sending data and one for downloading/getting data from internet). On the customer side, a splitter separates the data and telephone signals arriving to the home and forwards the data signal to the DSL modem(modulator/demodulator). On the telco side, in the CO(Local central office), a DSLAM(Digital Subscriber Line Access Multiplexer) separates the data and phone signals and sends the data to the Edge Router.

2- Cable Internet (HFC): It's common in home access. The cable television company transfers data to the edge routers using a combination of fiber cables and coaxial cables since the two kind of cables are used, this kind of access is also known as Hybrid Fiber Coaxial or HFC as well.In this kind of

access network data transference to and from houses happens using coaxial cables that then reach a main coaxial trunk. The coaxial trunk is then connected to a fiber node. The fiber node is connected to the cable television company's HQ using fiber cables. Inside the cable television company's HQ there exists a cable modem termination system (CMTS) that serves a conceptually similar purpose to DSLAM and does the process of turning analog signals taken from the cables back to digital ones. Instead of DSL Modems special cable modems provided by the cable television company are used. Its a shared broadcast medium.

3- FTTH: It's common in home access. There are two architectures for it's implementation known as Active Optical Networks (AON) and Passive Optical Networks (PON). We'll discuss PON for now.In PON, The teleco's CO offers the infrastructure in the form of fiber optics that reach a splitter and then separate and reach each home. In this kind of access network data transference to and from houses happen using an ONT (Optical Network Terminator) that's connected to the fiber optic for home which itself then reaches an Optical Splitter and then becomes the main trunk of fiber optics which itself reaches an Optical Line Terminator (OLT) in the teleco's CO, the OLT converts the optical signals into electrical ones and gives it to the edge router.

4- Dial Up: The dial up provider provides a phone number by calling which your Phone line will be used in an UNSIMILAR and "UN-Optimized" fashion from DSL, to send the digital data.

One Main topic to ask is: Why don’t

UML Subject 6:

Diagrams

Class Diagram

Diagram for showing classes (Serious stuff).

Class

It’s a rectangle divided to three compartments. Top represents class name. Middle Represents instance vars and bottom represents methods.

Constructor

To distinguish a constructor from the class’s oper ations, the UML requires that the word “constructor” be enclosed in guillemets (« and ») and placed before the constructor’s name.

– name : String

«constructor» Account(name: String)

+ setName(name: String)

+ getName() : String

Assosiation

2 way relation between 2 classes using pointer members.

Directed Assosiation

1 way relation between 2 classes using pointer members.

Aggregation

A class has members of another but those members's class are not dependent on it.

Composition

Used to introduce a class owned by another class.

The diamond is on the side that OWNs the other.

Generalization

Used to introduce a class that our current class is an identical type of. Inheritance.

“It’s the same but with some extra steps”

For abstract classes (as in Java) we use Generalization for depiction.

Dependency

Such as function arguments. Doesn’t own them or anything just momentary usage.

Interface Realization

Used to introduce an interface for an existing class

Use Case Diagram

Actor

An actor is what interacts with a system.

Use Case

Is an act performed by Actor. Depicted as **verb+noun**.

Use Case Subject

Is the system being interacted with.

Association

Basic relationship between an actor and use case.

Directed Association

Association but one sided.

Generalization

Inheritance but for use cases between use cases and actors.

Dependency

Depends on another.

Include

When user A happens B is committed as part of it.

Extend

A possible extension of an existing Use Case.

Files

Are sequences of bytes holding values. Based on the complexity of reading them they are grouped under two generic categories:

- Text Files: The bytes will be read thru and based on an encoding (UTF-8,ASCII,etc) as representatives of characters. They could have file formats

but it's not really necessary as they rely on a single encoding to do it for them.

- Binary files: The bytes COULD be text or literal values. Some parts MIGHT need to be read based on Encoding some others might be literal values meant to be read as numbers! They RELY on format to address this.

Constants

MessageBox Constants

Beginning with the prefix MB these constants which are used to set flags for MessageBox to change it's appearance are grouped in three self explanatory categories below for your comfort:

1- Button templates – Define the buttons that your Message box will have:

MB_OK 0x00000000L

MB_OKCANCEL 0x00000001L

MB_ABORTRETRYIGNORE 0x00000002L

MB_YESNOCANCEL 0x00000003L

MB_YESNO 0x00000004L

MB_RETRYCANCEL 0x00000005L

2- Default Button Setter – Define which button will be the default one **from left to right**
(The one that is selected by default):

MB_DEFBUTTON1 0x00000000L
MB_DEFBUTTON2 0x00000100L
MB_DEFBUTTON3 0x00000200L
MB_DEFBUTTON4 0x00000300L

3- Icon setters – Define the Icon within MessageBox:

MB_ICONHAND 0x00000010L
MB_ICONQUESTION 0x00000020L

MB_ICONEXCLAMATION 0x00000030L
MB_ICONASTERISK 0x00000040L
these icon constants do have alternate names as well:
MB_ICONWARNING MB_ICONEXCLAMATION
MB_ICONERROR MB_ICONHAND
MB_ICONINFORMATION MB_ICONASTERISK
MB_ICONSTOP MB_ICONHAND

Types

In order to have a level of compiler-implementation independency Microsoft decided to define their own types,instead of direct reliance on compiler types.

HINSTANCE: A pointer sized datatype used to keep track of the handles.

LPSTR / PSTR: A pointer to an array of 8-bit characters, which MAY be terminated by a null character.

Questions to ask AI

Witcher 2 Binary investigation

It seems while .text section provides close to no clues, as we doom-scroll upward since a certain point we start seeing things like GetModuleHandleW, OpenMutexW, CreateMutexW, R CloseHandle KERNEL32.dll, GetDlgItem, , DialogBoxParamW,SetWindowTextW, wsprintfW, SHELL32.dll and etc.

In another place I can literally see literal piles of C++ code defining a namespace “and hkHavok%Classes”

RE3 Remake Investigation

Upon reaching mscorelib things start to look readable.

execommon.inl

A file loaded in visual studio release configuration while using break points.

Visual Studio Sentience

Visual studio seems to be aware of the location of the C files of a static library built as a separate project of the same solution, the actual lib file of which I include in another project within the same solution IN different folders. This happens when I use VS in release mode with breakpoints. It loads the said C file.

References

1- C How to Program by Dietel brothers

2- C++ How to Program by Dietel brothers

3- Programming Windows by Charles Petzold

4- x86 Assembly by Kip Irvine

The list will be added upon

Some of the BEST books the existence of which had I been aware of , I might’ve reconsidered writing significant portions of this book:

Computer Systems a Programmer’s Perspective

Thinking in C++ Bruce Ekel

Subject 4: OpenGL

Open GL as a standard

Open GL before becoming a standard was an actual closed source library, that then formed a generic framework for graphics rendering.

As a standard open gl is distributed in 2 ways:

- Specifications: for it’s versions, stating the fundamental and foundational functionality that they are expected to cover

- Extensions: Additional functionality that’s not part of the official OpenGL.

Flow

OpenGL is implemented within the driver of GPUs as dynamic libraries, the static reference library and function declarations header of which is typically provided as a part of OS's sdk (at least in the case of windows).

Specification

Sets a set of basic functions that are to be provided within the implementation

Open GL Has a core specification and then it has the extensions, essentially features which were not part of the core Open GL but added as extension by the vendors.

ARB extensions are essentially extensions that are rather standardlized.

Windows

In Windows the general process of connecting to Open GL could be summarized like this:

1- Upon calling the wglCreateContext the windows looks for it's implementation in the OpenGL32.dll (The Windows Interface)(Naturally you must link to OpenGL32.lib to be able to use the DLL so don't forget to set it in compiler settings).

2- The OpenGL32.dll ends up calling other functions in the process, including the D3DKMTQueryAdapterInfo(,,,) and other functions,learning about the ICD location and implementation and what and how to expect.
3- The ICD DLL gets loaded.
4- Function addresses get initialized accordingly.

Context Creation

The process of connecting the a windows window to the OpenGL ICD thru the OS API's function is called OpenGL context creation. It is the first step for OpenGL programming. Generally context creation for OpenGL could happen in two ways:

1- **The "classic" /"default context" way**
2- **The modern way through extensions**

Regardless of the kind of context the context itself gets assigned per thread.

(Windows Kernell keeps track of the process IDs and their threads as it's doing its thing, all in memory)

1.The Classic WGL Context

In this way the default and vendor decided profile and version of Open GL is connected to your window. It involves:

1- We must set the pixel format for our hdc using **SetPixelFormat()** first.
2- It involves getting the window's HDC choosing and setting the pixel format,then calling wglCreateContext and wglMakeCurrent. This only gives

the old compatibility style context no specific versions requested but wglGetProcAddress, call wglCreateContextAttribs to request Open GL version and profiles. And delete the temporary one.

Using a context is essentially the easy "just load whatever default context is available for the gpu" way to have your initial context.

2. The Modern OpenGL Context

In order for our non classic manual context creation to be possible we have to use ARB functions… which themselves ought to run within an existing OpenGL context. As a result the manual context creation demands that we have a context already which is exactly why we create a default classic context first. These ARB functions themselves are essentially Open GL extensions but strong ones that are convention ed as the standardalized extension contract for providing the functions for bridging OpenGL contexts to windows.

For Windows that standardalized extension for window context creation is provided within the header wgl.h

In this way wglCreateContext alone is used to create a default temporary context and then using functions the actual version of choice (if available) gets loaded, set and the old temporary context gets removed.

for the modern context we must first create a temporary context first just to load extension then proceed to get the address of wglCreateContext attribs using.

And then use the function wglCreateContextAttribsARB()

Classic WGL Context

Binary Files

Binary executables machine code on mainstream OSs rely on an un obfuscatable basic skeleton that NEEDs to have the sections .Code,.Data and etc in one form or another while the symbols could be and usually are obfuscated.

This basic structure and the limited number of instructions in Assembly are why Disassembly seems a lot easier and is usually done by Dump apps (such as dumpbin/objdump). All these disassemblers are basically doing is:

1- Recognizing the opcodes (instructions that tell CPU to do stuff) from the values within the machine instructions.

2- Replacing the opcodes with their equal-in-context mnemonics in assembly.
3- Changing the format to that of Assembly instruction (such as AT&T or MASM).

In the case of PE files they may or may not contain machine language, the doctrine for file formats defines the section there must be not the type of data, the way they are represented may or may not allow for much flexibility.

Terms

Chapter 3: Visual Studio IDE stuff

Preprocessors

The default preprocessor inclusions _UNICODE and UNICODE are inherited by IDE by default.

Debugger

Most C/C++ IDEs have a debugger that allows adding breakpoints within software code for checking instruction and variable's values in a Local or watch section. Within the Watch section the domain of what you see is usually vast, in the local section you see the local function vars…usually only when in the debug configuration tho, while some control is still available in the release configuration in order to be able to observe each function step by step and see the local vars you NEED to be in debug config.

Endianness

Naturally the Debuggers tend to abstract away the endianness, showing the representation of the values AFTER applying the endianness.

Chapter 4: OpenGL Graphics Programming

OpenGL is a graphics programming API/standard offering a basic pipeline and a set of basic tools for achieving graphics programming.

Chapter 5: Binary Analysis in Linux

Section 1: Formats

ELF Format

The binary file format is used for both executable (.out) and relocatable (.o) object files generated by the C compilers (at least gcc). Either way they include some basic sections, remarkably a .text section for all the codes and a .data section for …well identification data I reckon. While in the non stripped version things seem rather clear…At least relatively,in the non stripped version at least for now the best

way to find and identify functions seems to be reliance on the address space they had pre stripping.

.init section

For initializing the program stub and the like.

.text section

It contains the machine code of the program.In the non-stripped output tends to contain all the functions and their names in the <functioname> format.

_start function seems to set the command line arguments.

Endbr64 mysteriously marks the beginning of the functions and alongside the beginning address of the functions in non stripped version, could be used to find them in the stripped version.

DWARF Format

Another format used for debug information. It’s usually embedded within ELF files but could be separate as well.

Section 2: The Loading Routine

Loading Programs

When a program is run, the OS makes a Process for it and assigns a virtual address space and an interpreter to that process.

Section 3: Tools

Text Reader

It just prints the bytes as the encoding that it supports.

cat filename.ext

It prints the bytes as ASCII characters

.

Dump Readers

These apps are used to specifically real the machine code sections **(show the actual bytes)** of a format and usually disassemble them and give VALUES.

objdump

It's a dump reader used to **show the actual bytes.**

objdump -s filename.ext

Used to show the bytes of the different sections of the file in text editor-ish manner.

objdump -sj sectionname filename.ext

Used to show the bytes of the specific section of the file in text editor-ish manner.

objdump -d filename.ext

Used to disassemble the .text section of the file.

Format Readers

These apps basically just read the file with the intent of showing the stuff such as symbols and general informative, locational stuff in a Format. Mostly to give information alone.

readelf

It's an elf reader. It's commands are used **to load and give the attributes** of ELF file formats (mainly it's data section).

readelf -a filename.ext

Used to load ALL the info about the elf.

readelf --syms filename.ext

Used to load the info about the symbols section of the elf file.

readelf--relocs filename.ext

Used to load relocation info about the ELF file.

Chapter 6: Binary Analysis in Windows

Chapter 7: Internationalization of programs

Chapter 8: Android Development

CSFOA: ***Conditional Symbolic Function-Overload Abstraction.***

.****** ******* ******* *******.****** ******* ******* ******.****** ******* ******** ******* Generic methods: ******* ******* ******** ******** We can limit the typeparameter in class using notion.********

Java left at multi dimensional arrays.273Book/317 PDF.

Computer Systems left at machine language.

YOOOO

Stuff to add:

The ARB function used to set version.

The delete keyword usage in C++ how it makes things unavailable, usually as an attempt to keep compiler generated stuff from becoming accessible.

The way WinProc adds a new message to queue upon closing it.

In Windows do NOT fuck with currentDirectory all that much. Instead rely on setting a main application root using a misleading-by-name function named GetModuleFileName() to achieve your desired effect. YOU STILL CONSTRUCT THE WHOLE path it's not done RELATIVE THE CURRENT WORKING DIRECTORY.

Reinterpret_cast<whatever> (whatev) is essentially used to get us the same level of type unsafety ness we had with pointers in C.

Used GetModuleFileName(nullptr, array, MAX_PATH) to get module directory including the module file itself.

Then proceeded to use PathRemoveFileSpec to remove the module file name and it's previous backslash.

Don't forget to ask regarding the different roles of the headers that you had to download.